R Best Practices

Nicholas Tierney

2026-02-17

Table of contents

Welcome

Welcome to R Best Practices! This course covers essential best practices for writing clear, maintainable, and reproducible R code.

Course materials

Materials will be added here as we progress through the course.

https://r-best-practices.njtierney.com

Prerequisites

- Basic R programming experience
- Familiarity with writing R scripts
- Experience working on data analysis projects

Learning outcomes

- How to name things effectively
- Using a style guide
- How to refactor your code
- How to review your code and others'
- How to lay out a project so others know how to run your code
- How to make a reproducible example (reprex)

Schedule

Project Organisation

- Understanding file paths
- Pitfalls of organisation
- Common project structures
- Naming files
- "Good Enough"/Common Sense principles of project organisation
- Always have a README
- RStudio and positron set up

Code style and readability

- Style is like grammar
- Layout, naming, and correctness are three different jobs
- Why we care about consistent names, spaces, and indentation
- Using the {air} formatter
- Using linters
- Bonus: typing skills and keyboard shortcuts

Writing readable code

- Where we are up to: what the tools fixed, and what they didn't
- Good vs bad variable names
- De-chunking code
- What to look for: code smells, inputs and outputs, complexity, refactoring
- A process for reviewing code
- Reviewing someone else's code
- Clean as you go

Writing Functions

- The problem functions solve
- Anatomy of a function
- Good (simple) function design
- Using {fnmate} to speed up creating functions
- How to use a debugger

Writing a reprex / getting unstuck

- How to share problems
- Using {reprex}
- Practicing reprex
- Practical tips on debugging
    - Keep solving vs cleaning up

Putting It All Together

- Take an existing project and provide code review
- Apply code linting, code styling, functions
- Discussion and Q & A

Setup

Two things before we start:

1. How this book is put together, and
2. What to install.

The first part takes two minutes to read and will make the rest of the course easier to follow.

How this book works

The book is longer than the course

The course is six hours. The book has more in it than six hours can cover, and that is on purpose.

We follow one path through it live, and the rest stays here for when you want it. So if we skip a section, or if you notice a box we never opened, nothing has gone wrong - it is there for you to come back to. I would rather write the complete version and choose what to teach on the day than write only what fits and leave you with notes that stop where the session did.

How we will work

Mostly by typing. I will write code, you write it too, and we will break things together and then fix them. You will get more out of this by typing the examples than by watching me type them, even when it feels slower.

You do not need to keep up with every keystroke. Everything on screen is in the book, with a copy button on every code block.

The boxes

Four kinds of box turn up throughout, each with its own colour and icon:

- **Your Turn** - something for you to do. Most of these we do together in the session, and a few are there for afterwards. The ones that ask what you think have no wrong answers; I am genuinely curious what has gone wrong for you before.

- **Some history** - where something came from. Why a linter is called a linter, where the backslash in `\(x)` comes from. Entirely skippable, and quietly my favourite part of the book.
- **Going deeper** - optional extra detail, a longer example, or a list you might want later. Boxes titled **Read more** point at the blog post or talk a section is adapted from, so you can follow the original if you want it.
- - the things that will actually bite you. There are not many, and that is deliberate.

Rather than describe them, here is one of each.

Your Turn

Exercises look like this. Most of them give you something to run:

```
R.version.string
```

Some of them just ask you something. Here is the one I actually care about: what is the worst R code you have ever had to work with? Your own counts, and mine definitely does.
There is no wrong answer. I am asking because the answer usually tells me which parts of the day to slow down on.

Some history: where the name R comes from

Two things at once, which I think is lovely.
R was written by Ross Ihaka and Robert Gentleman at the University of Auckland in the early 1990s, and they share a first initial. It is also a play on S, the language it was modelled on, in the same way that C came after B.
So the name is a pun and a signature at the same time. You can read their own account of it in R: A Language for Data Analysis and Graphics.

Going deeper: how these boxes are made

You just clicked a title, so that is the collapsing demonstrated. These are Quarto callouts, and there is nothing clever going on. Each one is a fenced div with a class on it:

```
::: {.callout-note title="Your Turn"}

Something for you to do.

:::
```

The colours and the icons come from a stylesheet in the book's repo, not from Quarto. Quarto ships five callout types with its own icons, and I have repainted all five.
If you want them in your own work, the docs are at Quarto: callout

blocks.

! Restart R after a big install

If you install a package that is already loaded in your session, R can end up half on the old version and half on the new one. You then get errors that make no sense and do not survive a restart, which is a miserable way to spend twenty minutes.
So once you have worked through the installs below, restart R. In RStudio that is `Cmd / Ctrl + Shift + F10`.

Some boxes are collapsed, showing only their title, like the **Going deeper** one above. Click the title to open one. If you are reading the PDF they are all open already, which is one reason the PDF is longer than it looks online.

The code

Code blocks look like this:

```
library(tidyverse)

airquality |>
  count(Month)
#>   Month  n
#> 1     5 31
#> 2     6 30
#> 3     7 31
#> 4     8 31
#> 5     9 30
```

Lines starting with `#>` are **output**, not code. You do not type those - they are what R prints back, shown so you can check you got the same thing.

Longer blocks have line numbers in the margin so that I can say "look at line 4" and we are both looking at the same line. Short blocks do not, because there is nothing to point at.

Most examples use data that comes with R, like `airquality` above, so you can run anything here without downloading a dataset first.

We use the base pipe, `|>`, rather than `%>%` throughout. If you are used to `%>%`, everything here works the same way.

Where everything lives

- **This book** - https://r-best-practices.njtierney.com, and there is a PDF download in the sidebar if you would rather print it or annotate it.

- **The book's source** - https://github.com/njtierney/rbestpractices. Every page has an "Edit this page" link, which goes straight to the file that made it.
- **The books exercises** - https://github.com/njtierney/rbp-exercises. We will discuss how to "automagically" open this in the course

If you find a mistake, please tell me. Every page has a link to open an issue, or you can email me. Errors in teaching material are worth more than errors in almost anything else I write, because thirty people hit them at once. Finding one is a favour, not a complaint.

What to install

If you can, it is worthwhile to install the below before the first session.

We have time on the day, and I would much rather spend ten minutes helping you get set up than have you sit out an exercise. So work through what you can, and bring the rest along.

R

You need **R 4.5.0 or later**, from https://cran.r-project.org/. Check what you have with:

```
R.version.string
#> [1] "R version 4.6.1 (2026-06-24)"
```

The reason for 4.5.0 specifically is the penguins. From that version, R ships a `penguins` dataset with base R, so a few of the examples work without installing anything at all. Before 4.5.0, you had to get it from a package.

You also need an editor - the thing you use to edit your R code.

Personally, I recommend RStudio.

If you like, you can use Positron - for this course I mostly focus on RStudio, but am able to discuss some of the features with positron, which is posit(the company that made RStudio)'s latest generation tool.

If you already have one and you like it, stay where you are.

> If you are on an older R
>
> Everything in the course still works, but you will need the palmerpenguins package for the handful of examples that use `penguins`:
>
> ```
> install.packages("palmerpenguins")
> library(palmerpenguins)
> ```

> One catch: **the column names are different.** When the data moved into base R the names were shortened, so you will need to translate:
>
> | In this book | In palmerpenguins |
> |---|---|
> | `bill_len` | `bill_length_mm` |
> | `bill_dep` | `bill_depth_mm` |
> | `flipper_len` | `flipper_length_mm` |
> | `body_mass` | `body_mass_g` |
>
> The units left the names but not the data - `body_mass` is still in grams.
> I wrote about this and the rest of what landed in 4.5.0 here: R version 4.5.0 is out.
> You can use Ella Kaye's `basepenguins` R package to help you move your existing scripts across if you want to use base R penguins.
> Separately, some examples read a `penguins.csv` file rather than the dataset. Those use the longer names, because that is what is in the file - not because of your R version.

The R packages

It might will take a few minutes:

```
install.packages(c(
  "tidyverse",
  "here",
  "lintr",
  "styler",
  "spelling",
  "usethis",
  "reprex",
  "goodpractice",
  "sf",
  "pak",
  "sp"
))

install.packages('fnmate', repos =
 ↪  c('https://milesmcbain.r-universe.dev',
 ↪  'https://cloud.r-project.org'))
```

`sf` and `sp` are only needed for one optional exercises. They are the heaviest things on the list, so if they give you trouble, leave them and let me know.

Code formatters

Air is an R formatter from Posit. It is not an R package, so it installs

from the terminal rather than from R.

See their installation instruction on their website:

But briefly, here are the most popular ways to install:

On macOS and Linux:

```
curl -LsSf https://github.com/posit-dev/air/releases/latest/d
↪  ownload/air-installer.sh | sh
```

On Windows:

```
powershell -ExecutionPolicy Bypass -c "irm https://github.com
↪  /posit-dev/air/releases/latest/download/air-installer.ps1
↪  | iex"
```

Check it worked:

```
air --version
```

You should get a version number back. If you get "command not found", the install did not finish, so try it again.

We set up format-on-save together in chapter 2, so there is nothing else to do with air before we start.

If you have trouble installing {air}, `{styler}` is in the package list above and does the same job, just not as fast.

Jarl, the linter

Jarl is a fast linter by Etienne Bacher, built on top of Air. Installation instructions are on its site.

Jarl is optional. Like air, it is a command line tool rather than an R package, so the install can go wrong in all the same ways, and I do not want you fighting it before we start.

If it gives you any trouble, do not worry about it. `{flir}` is in the package list above - it is by Etienne as well, it installs the way every other R package installs, and it is the one that will still fix things for you. `{lintr}` is in the list too. Everything in chapter 2 works with any of the three.

Checking it all worked

Run this in R. It should print `TRUE` for everything.

```
pkgs <- c(
  "tidyverse", "here", "lintr", "styler",
  "spelling", "usethis", "reprex", "fnmate",
  "flir", "goodpractice", "sf", "sp"
)

sapply(pkgs, requireNamespace, quietly = TRUE)
```

Then, in the terminal:

```
air --version
```

If both of those look right, you are set.

If something will not install

Please do send me an email before the session rather than fighting with it on your own.

Installation problems are almost always specific to one machine, and they are much faster to solve with two people looking at them. They are also, genuinely, not a reflection on you. Every one of us has lost an afternoon to a package that would not build. Ask me for some horror stories.

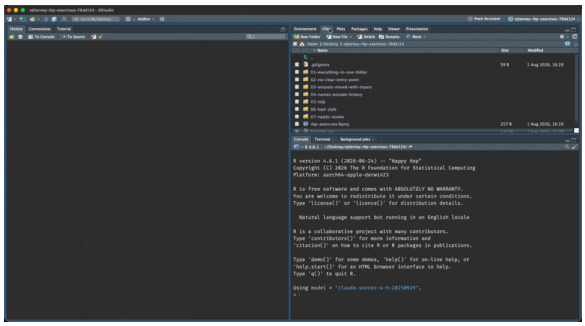
Get course exercise materials on your machine

Your Turn

Let's download the course exercise - try running this code:

```
use_course("njtierney/rbp-exercises")
```

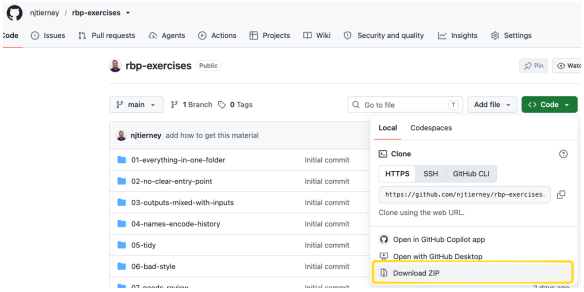
Which will prompt you to download the repository, https://github.com/njtierney/rbp-exercises, and then open it in an RStudio project.



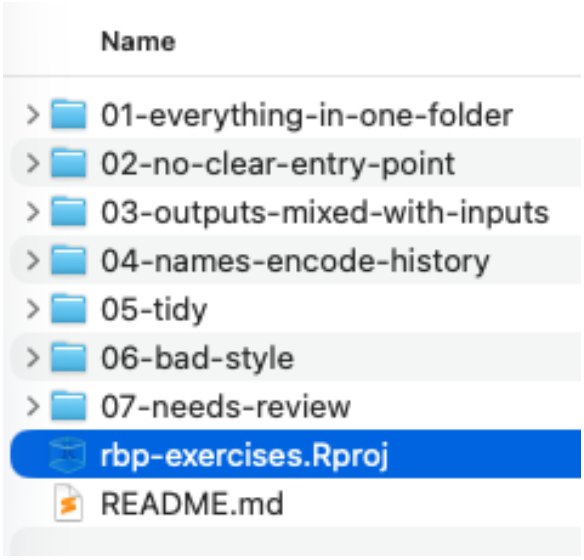
Where you end up. This one is a screen recording in the web version of the book.

If you have troubles with this (sometimes firewalls can stop you)

you can download it from the github page https://github.com/njtierney/rbp-exercises like so:



Then, unzip that, and click on the rbp-exercises.Rproj file:



1 Project organisation

> "Remember, you are always collaborating with your future self."

This course is all about best practices in R. At its foundation, I think this begins with a well organised project that is clean and tidy. We start by talking about basic "hygiene" for your projects and files, before moving on to how to organise your project, and some potential pitfalls.

Remember: your project doesn't have to be perfectly organised. Instead, we focus on making the next project a little better, each time.

It is worth noting that this section draws from "Workflow" and RStudio, what and why from my course: "quarto for scientists".

Overview

Duration 60 minutes
Questions

- How should I set up RStudio or Positron?
- What is a file path, and why does `setwd()` break things?
- What is a project, anyway?
- How does `{here}` find my files?
- Why should every project have a README?
- What goes wrong when a project is not organised?
- What does "good enough" project organisation look like?
- How should I name my files?

1.1 Set up

Go to course exercises and follow instructions to download course materials.

Your Turn

When you start a **new** project:

- What is your normal workflow?
- What helps you, what hinders you?
- What do you think you should do?

When you come back to an **old** project:

- What has made it easier to work on it?
- What makes it difficult?

- Anything you wish you did?

There are no wrong answers! I want to have a discussion, and learn what wins, what makes it hard, what makes it OK.

Let's talk about some easy-wins in how you set up RStudio and Positron, to make it easier to make things reproducible.

1.1.1 RStudio

We want to go to global options, like so:

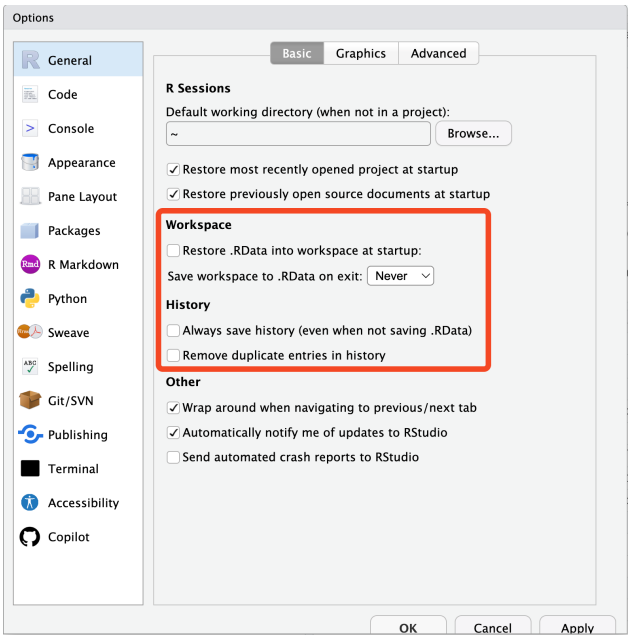
Tools → Global Options (or `Cmd + ,` on macOS) → General.

Under **Workspace**:

- Uncheck **Restore .RData into workspace at startup**
- Set **Save workspace to .RData on exit** to **Never**

Under **History**:

- Uncheck **Always save history (even when not saving .RData)**
- Uncheck **Remove duplicate entries in history**



Setting the options so you neither restore nor save your previous session's work.

This matters for two reasons:

1. **Reproducibility.** You don't want old objects from your last analysis cluttering your session - and you also don't want to depend on them, accidentally.

2. **Privacy.** You really do not want private data written into a `.RData` file on disk. You want to read data in, deliberately, every time.

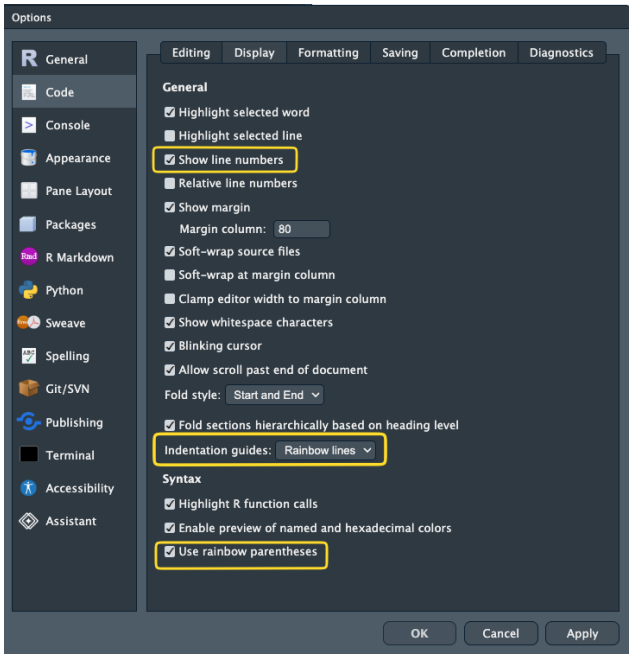
Your history is the list of commands you have typed. Not saving it means you won't come to rely on something you typed in a previous session - which is a good habit to build.

You can also do establish these settings via the `{usethis}` package:

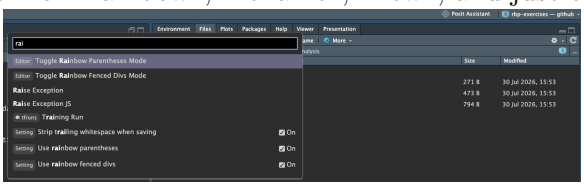
```
usethis::use_blank_slate()
```

While you are in the settings, two more worth turning on:

- **Show line numbers** - Easier to say: "The code on line 54"
- **Rainbow parentheses** - Catch those missing brackets
- **Rainbow indentations** - Fun indentation markers



> **Make a habit**
>
> You can restart R with a keyboard shortcut:
>
> - **Restart R**: `Cmd/Ctrl + Shift + F10`
> - Your R session will always start from scratch now.
> - Make a habit of restarting R and starting from scratch.
> - If your code works from a fresh session, that is a great standard
> - If it errors on a fresh session, you've got a problem to solve (which is overall a good thing to identify!)

> **Your Turn**
>
> There is a really neat feature in RStudio called the "Command Pallete" - it's kind of like "spotlight" or "search" for any setting in RStudio.
> Try opening it using the keyboard shortcut:
>
> - Windows: `Ctrl + Shift + P`
> - Mac: `Cmd + Shift + P`
>
> And search for "rainbow", "rename", "new", and just explore!
>
> 
>
>
> Read more at Posit's RStudio guide to the Command Palette

1.1.2 Positron

The same settings live under Settings → Extensions → R. The equivalent behaviour is off by default, so there is usually nothing to change - but it is worth looking, so you know where it is.

Positron also has the command palette! The same command as RStudio.

- Windows: `Ctrl + Shift + P`
- Mac: `Cmd + Shift + P`

> **Your Turn**
>
> 1. Turn off workspace saving using the settings above, or run `usethis::use_blank_slate()`.
> 2. Restart R with `Cmd/Ctrl + Shift + F10`.
> 3. Open your most recent analysis script and run it from the top in the clean session.
> 4. Discuss why your work did/did not work

> **RStudio vs Positron?**
>
> Worth saying upfront - RStudio isn't going away.
>
> - RStudio
>   - created in 2011.
>   - Focus on R, but allows you to write in Python.
>   - Supports other languages like C, C++.
>   - It has 15 years of design for use in data analysis. It just works.
> - Positron

  - Created in 2024.
  - A fork of "Visual Studio Code - Open Source"
  - Comes with many (most?) of the features of VS Code
  - Rich marketplace of extensions
  - "polyglot" - works with many languages, not just R

Which to choose? Here are my opinions:

- New to R –> RStudio
- Just using R –> RStudio
- Mostly using R, occasional other language –> RStudio
- Sometimes R, mostly other language –> Positron
- R + frequently other other languages (Python, Rust, C) –> Positron
- Experienced in R and want full control of every setting –> Positron

If you want to read more about the features of Positron and RStudio, see Posit's article on the topic.

1.2 A common problem: file paths

Before we get into projects, and organising them, I really think it is worthwhile spending a bit of time talking about **file paths**.

I can almost guarantee that you have run into this problem:

```
read.csv("my-very-important-data-file-somewhere.csv")
```

```
Warning in file(file, "rt"): cannot open file
'my-very-important-data-file-somewhere.csv': No such file or directory

Error in `file()`:
! cannot open the connection
```

This error is R saying:

> I do not know where "my-very-important-data-file-somewhere.csv" is.

There are lots of reasons that R doesn't know where that file could be!

Chances are **you** know exactly where the file is, and now you need to get into the mind of R, to understand **why it cannot find the file**.

We can resolve these errors if we keep our **files**, **paths**, and **directories** ordered. This goes together with a **workflow** for clearly, and portably letting R know where these are. This practice is sometimes referred to as **good file path hygiene**.

Let's define a few of these terms: files, directories, and paths.

1.2.1 Files, directories, paths

Below we show some examples of a folder structure, of directories, and files

```
Users/                                ← Directory
└── njtierney/                        ← Directory
    └── Desktop/                      ← Directory
        └── analysis-2026/            ← Directory
            ├── analysis-2026.Rproj   ← File
            ├── data/                 ← Directory
            │   └── penguins.csv      ← File
            ├── exploratory-analysis/ ← Directory
            │   ├── explore.R         ← File
            │   ├── eda-document.qmd  ← File
            │   ├── eda-document.docx ← File
            │   └── graph.png         ← File
            └── README.md             ← File
```

We can refer to a given location with a "path" - which we refer to as a "file path" or a "directory path" depending on which we are navigating to. Here are some examples of paths for two directories, and two files.

- `Users/njtierney/Desktop/analysis-2026/`
  - **Directory** path of "analysis-2026"
- `Users/njtierney/Desktop/analysis-2026/README.md`
  - **File** path of "README.md"
- `Users/njtierney/Desktop/analysis-2026/data`
  - **Directory** path of "data"
- `Users/njtierney/Desktop/analysis-2026/data/penguins.csv`
  - **File** path of "penguins.csv"

A path is just the folders you walk through to get there, joined up with `/`. Here is that last one again, with every step of it marked:

```
Users/                          ●
└── njtierney/                  ●
    └── Desktop/                ●
        └── analysis-2026/      ●
            ├── analysis-2026.Rproj
            ├── data/           ●
            │   └── penguins.csv●
            ├── exploratory-analysis/
            │   ├── explore.R
            │   ├── eda-document.qmd
            │   ├── eda-document.docx
            │   └── graph.png
            └── README.md
```

Read the marked lines top to bottom and you have written the path:

```
/Users/njtierney/Desktop/analysis-2026/data/penguins.csv
```

Nothing else in the tree is part of it. That is the whole idea - a path names one route down through the folders, and ignores everything it walks past.

Let's talk a bit more about files, directories and paths.

1.2.1.1 Files

- Files are something I can open.
- These are all files: "explore.R", "eda-document.docx", "graph.png", "README.md", "penguins.csv"
- Files have "file extensions", which are the last characters after a `.`, and they tell us what kind of file they are:
  - "explore.R" ends with **.R**, it is an R script
  - "eda-document.docx" is a Microsoft Word Document
  - "graph.png" is an image (specifically, a "PNG")
  - "README.md" is a **m**ark**d**own text file.
  - "penguins.csv" is a "comma separated values" file

1.2.1.2 Directories

- Also known as folders (files inside a folder)
- A directory contains files
- A directory can also contain directories

1.2.1.3 Paths

A path, or "file path", or "directory path" is the machine-readable directions to where files on your computer live. Kind of like a street address. "Machine readable" just means your computer can read it easily:

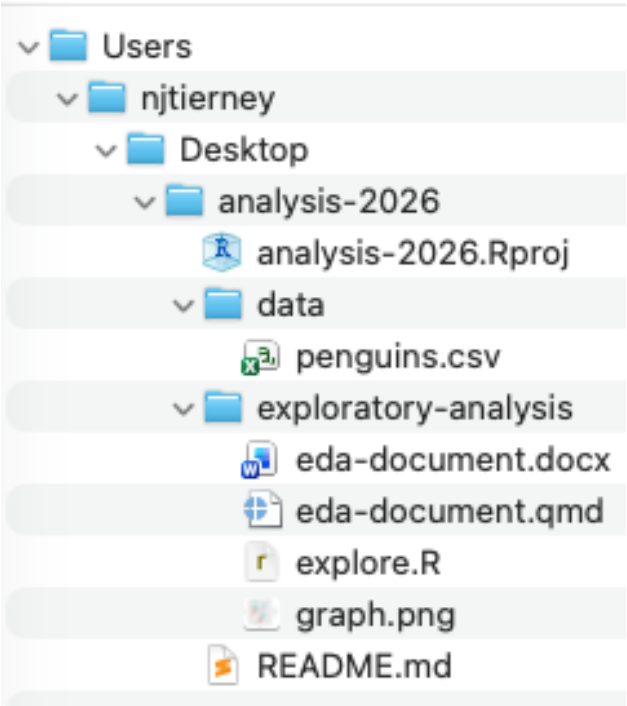
- Machine readable:
  - `/Users/njtierney/Desktop/analysis-2026/data/penguins.csv`
- Not-quite machine readable:
  - "The folder on my desktop named analysis 2026"

So, the **file path**:

```
/Users/njtierney/Desktop/analysis-2026/data/penguins.csv
```

Describes the location of `penguins.csv`

It might be easier to see this as a tree, or how this file might look in a file explorer on your computer:



1.2.2 Reading files

So to read CSV in R, you can put the path of penguins.csv in `read_csv()`:

```
penguins <- read_csv("/Users/njtierney/Desktop/analysis-2026/
↪  data/penguins.csv")
```

In our diagram, that is this file:

```
Users/
└── njtierney/
    └── Desktop/
        └── analysis-2026/
            ├── analysis-2026.Rproj
            ├── data/
            │   └── penguins.csv        ← this is the file being read in
            ├── exploratory-analysis/
            │   ├── explore.R
            │   ├── eda-document.qmd
            │   ├── eda-document.docx
            │   └── graph.png
            └── README.md
```

You might also note that sometimes in your work, you don't write down this full path - it is very long! You might do something like:

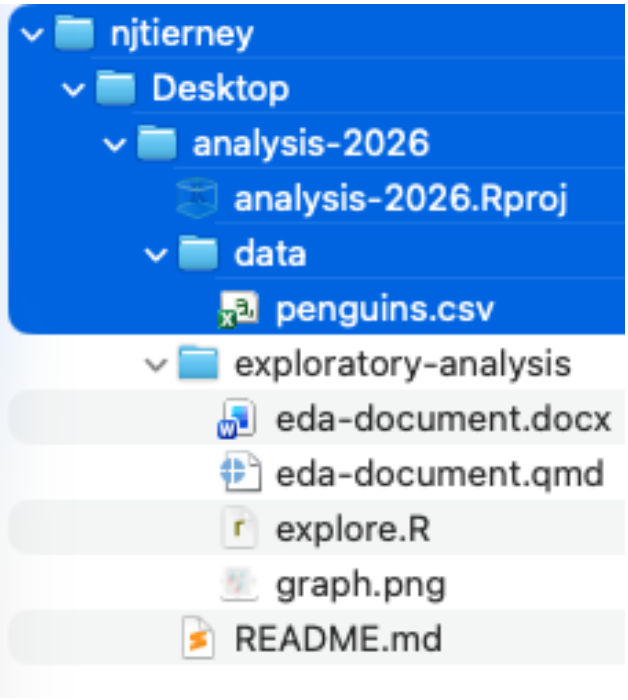
```
penguins <- read_csv("data/penguins.csv")
```

And this brings us to an important concept - these are two kinds of path:

- **absolute paths** starts from the root of your computer.
- **relative paths** starts from wherever you currently are.

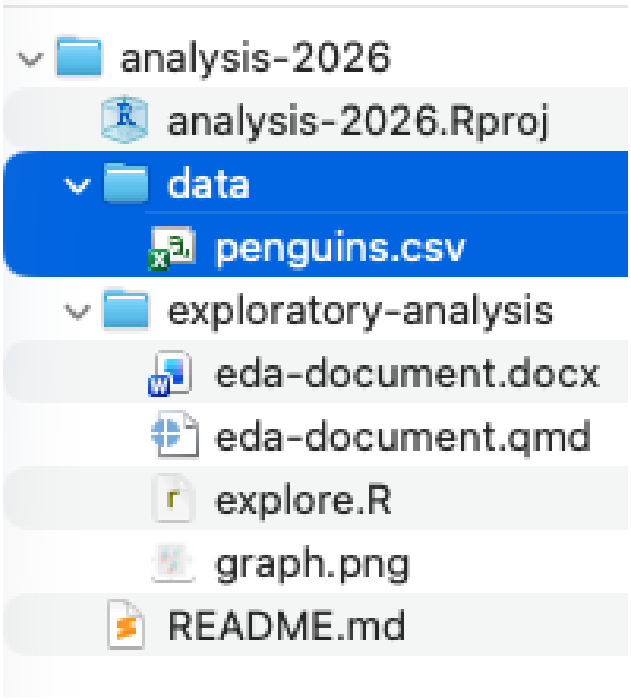
So, the code below is using an **absolute** path:

```
penguins <- read_csv("/Users/njtierney/Desktop/analysis-2026/
↪  data/penguins.csv")
```



The code below is using a **relative** path:

```
penguins <- read_csv("data/penguins.csv")
```



- The relative path looks nice and shorter. But it requires context - it needs to know from where you are reading.
- The absolute path will work from anywhere on your machine. It reads from the lowest folder on your machine.[1]

But, will the absolute work on your colleagues machine? On your machine in two weeks, next month, next year? No. My absolute path will not work on your machine. Your computer (almost certainly) doesn't have a `/Users/njtierney/` folder. It won't work on your own machine after you reorganise your folders when you spring clean. It won't work on the cloud.

> **Your Turn**
>
> 1. Imagine you see the path `/Users/miles/etc1010/week1/data/health.csv`. What are the folders above the file `health.csv`?
> 2. Run `getwd()`. Where are you?
> 3. Look at your most recent analysis script. Find every file path in it. How many are absolute? How many are relative?

A common answer to this is that people will often use relative paths, along with a function `setwd()`:

```
setwd("/Users/njtierney/Desktop/analysis-2026")
penguins <- read_csv("data/penguins.csv")
```

[1] That lowest folder is called the **root** - the one folder that contains everything else. On macOS and Linux it is written `/`, so an absolute path always starts with a `/`. On Windows it is usually `C:\`. Your own home folder sits just inside it, at `/Users/yourname` on macOS or `/home/yourname` on Linux, and has a shorthand: `~`. So `~/Desktop/analysis-2026` and `/Users/njtierney/Desktop/analysis-2026` are the same place, on my machine only.

Let's talk about why this is **not a good idea**.

1.2.3 What does `setwd()` do?

Sometimes the first line of a script looks like this:

```
setwd("/Users/njtierney/Desktop/analysis-2026")
```

This says:

> Set my working directory to this specific directory, "analysis-2026", which is in the folders, Users/njtierney/Desktop.

It means you can then read data using the **relative paths** we discussed above:

```
penguins <- read_csv("data/penguins.csv")
```

instead of using the **absolute path**:

```
penguins <- read_csv("/Users/njtierney/Desktop/analysis-2026/
 ↪  data/penguins.csv")
```

So it does have the effect of **making the file paths work in your file**.

But, it is still a problem, because using `setwd()` like this creates the same problem you had when you used the **absolute path**:

- 0% chance of working on someone else's machine - **and this could include you, in six months**.
- Means your file is not self-contained or portable. What happens if this folder moves to `/Downloads`, or onto another machine?

I used to send files to people like this, and you might have come across this yourself - where an R file comes to you and your colleague says:

> Yup, all you need to do is **change the `setwd("/Users/njtierney/Desktop/analysis-2026")` to point at where you are, then run it**

So, to get it working, you have to hand-edit the file path for every machine it lands on.

This is painful. And when you do it all the time, it gets old, fast. There is a better way!

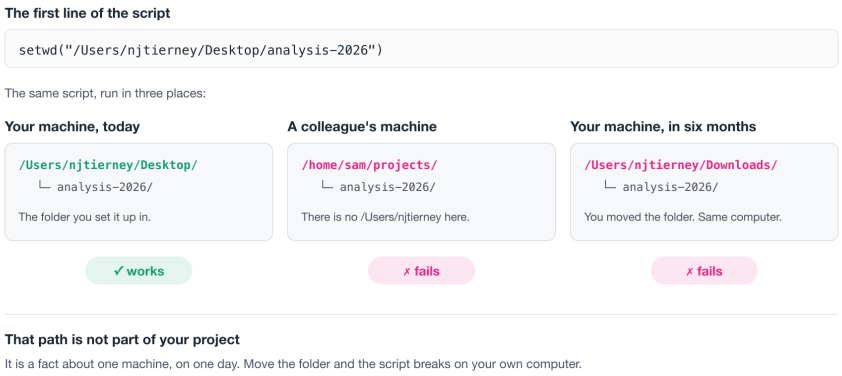


Figure 1.1: The path inside `setwd()` names a folder that only one machine has, and will not work in future places.

This is the heart of the famous line from Jenny Bryan, from her brilliant article, "Project-oriented workflow"- you should read it:

> If the first line of your R script is
>
> ```
> setwd("C:\Users\jenny\path\that\only\I\have")
> ```
>
> I will come into your office and SET YOUR COMPUTER ON FIRE .

Let's talk about using projects

1.3 Project oriented workflow

When you start on a new idea, a new project, a new paper - anything new - it should start its life as a **project**. In RStudio that is an `.Rproj` file; in Positron it is a **workspace folder**.

A project keeps related work together in the same place. They also do the following:

- Set the working directory to the project directory.
- Start a new session of R.
- Restore previously edited files into your editor tabs.
- Let you have several projects open at once.

This helps keep you sane, because:

- Your projects are independent.
- You can work on different projects at the same time.
- Objects and functions you create in one project won't affect another.

Projects resolve file path problems, because they automatically set the working directory to the location of the project.

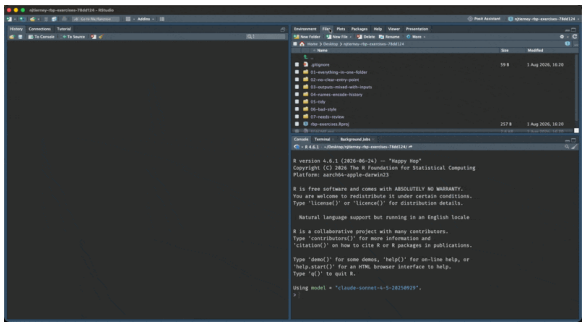
1.4 Get course exercise materials on your machine

Your Turn

Let's download the course exercise - try running this code:

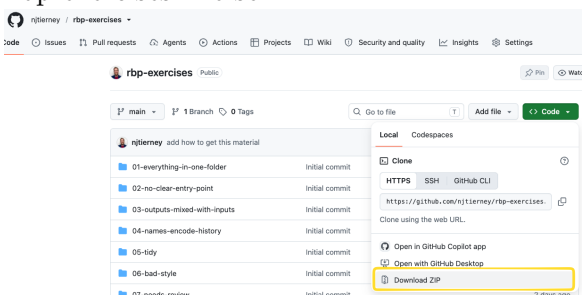
```
usethis::use_course("njtierney/rbp-exercises")
```

Which will prompt you to download the repository, https://github.com/njtierney/rbp-exercises, and then open it in an RStudio project.

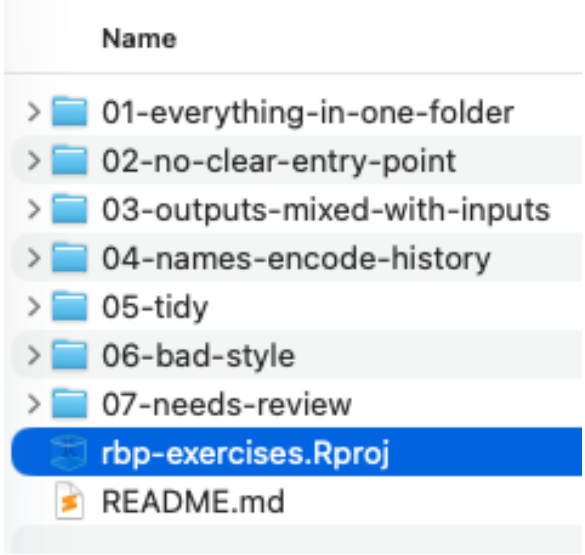


Where you end up. This one is a screen recording in the web version of the book.

If you have troubles with this (sometimes firewalls can stop you) you can download it from the github page https://github.com/njtierney/rbp-exercises like so:



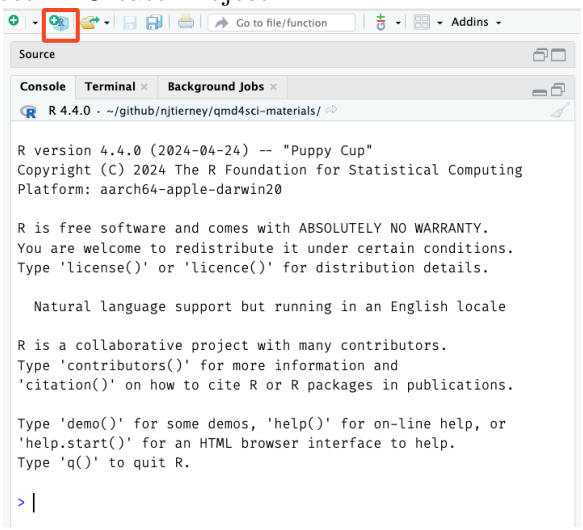
Then, unzip that, and click on the rbp-exercises.Rproj file:



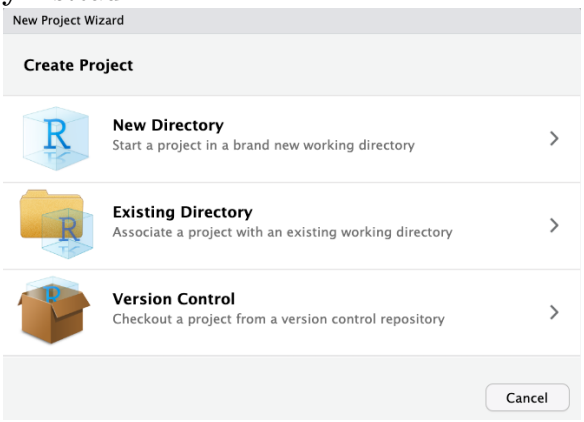
Aside: creating a project

1.5 RStudio

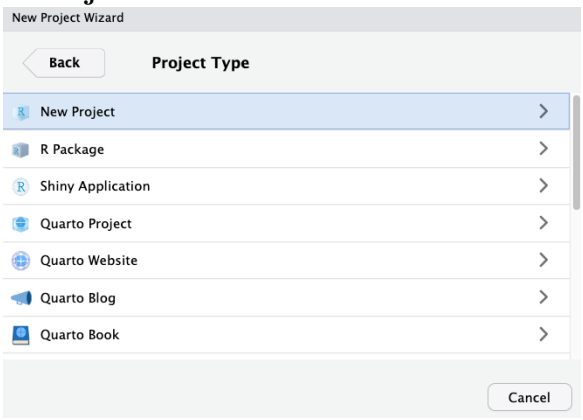
Either use the button in the top right corner, or go:
File → New Project → New Directory → New Project → name your project → Create Project



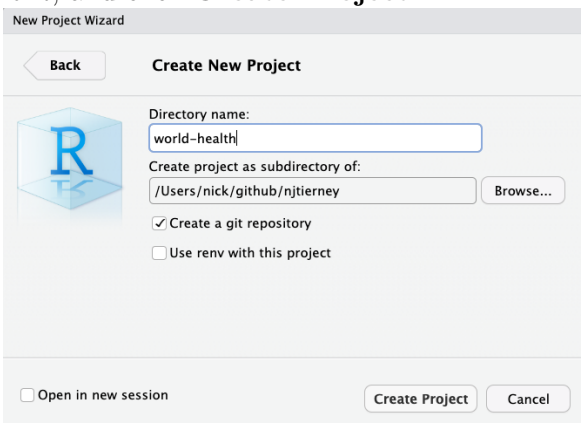
Starting a new project in RStudio.
Then choose **New Directory** if this is a new folder. If you already have a folder you want to turn into a project, choose **Existing Directory** instead.



Choosing a new or existing directory.
Then **New Project**.



Choosing the project type.
Then name it, and click **Create Project**.



Naming the project.

1.6 Positron

Positron doesn't use .Rproj files - that is an RStudio specific file. The posit team talk about this in their docs: "The R Proj File". The gist of this is that Positron does not have Rproj files - you essentially just open a folder and Positron will treat it as a "workspace". You can open a folder as a workspace: File -> Open Folder.
If you also want the project to work in RStudio, add an `.Rproj` file:

```
usethis::use_rstudio()
```

Either way, it is worth spending a couple of minutes on the name. Even a few minutes can make a difference. You want to:

- Keep it short.
- Use no spaces.
- Combine words.

For example, I had a project looking at bat calls, so I called it `screech`, because bats make a screech-y noise. If you are doing global health analysis, maybe it is `world-health`.

1.7 The {here} package

Although projects resolve most file path problems, in some cases you might have many nested folders. To navigate them reliably you can use the {here} package, which builds the full path from the project root, compactly.

```
here::here("data")
#> [1] "/Users/njtierney/github/njtierney/analysis-2026/data"
```

and

```
here::here("data", "penguins.csv")
#> [1] "/Users/njtierney/github/njtierney/analysis-2026/data/
 ↪  penguins.csv"
```

Note that these absolute paths will be different on my computer compared to yours - which is exactly what I want you to see.

You can read that `here` code as:

> In the folder `data`, there is a file called `penguins.csv`. Can you please give me the full path to that file?

This is handy for a few reasons:

1. It makes things *completely* portable.
2. Quarto documents have their own way of looking for files, and this eliminates a whole class of file path pain.
3. If you decide not to use projects, you still have code that works on *any* machine.

So in practice:

```
library(here)
penguins <- read_csv(here("data", "penguins.csv"))
```

works whether the code is called from the top of the project, from inside `analysis/`, or from a Quarto document being rendered in a subfolder - all places where a plain relative path can quietly mean something different.

> **Your Turn**
>
> 1. Run `here::here()`. Is it where you expected?
> 2. Open another rstudio project. Take one absolute path from your most recent script and rewrite it with `here()`.
> 3. What would have to be true about your working directory for the old version to work?

1.8 Organise your project with a README

When you are working on a project, you are almost certainly going to be returning to it, at some point. When you do, you might look at the code, and wonder:

> Who wrote this, and are they insane?

And then you realise that **you wrote this**.

I think it is worthwhile internalising this saying:

> You are always collaborating with your future self.[2]

I think a really easy win here is to have a README file, which is a plain text file that helps answer:

- **Why** you did it: what was the goal
- **When** you did it: Last year, last month?
- **What** you did: What was run
- **How** you did it: What was the order you ran things? Which file should I look at first?
- **Who** did it: Was it just you? Were there many people?
- **Where** you did it: On your computer? The cloud?

If you can't answer these questions about a project, how much harder would it be for you to return to it?

I like to think of this as some self-care for your future self.

What I am getting at here is:

> **A project that is organised is a project that's more likely to re-run.**

A well organised project does not guarantee reproducibility. But, if you can't tell me which script to run first, then we cannot reproduce an analysis.

A well organised project makes it easier to check, run later, and understand.

A README does not need to be long. Mine are often ten lines. It just has to answer those six questions.

Here is a complete, entirely adequate README, with the question each part is answering marked in the margin:

```
# Penguin body mass analysis                          ← WHAT

Fits a model of penguin body mass against flipper
length and species, for the 2026 report to the
department.                                           ← WHY

Run by Nick Tierney. Last run 2026-03-14, on my
laptop, with R 4.5.0.                                 ← WHO,
 ↪  WHEN, WHERE

## How to run                                         ← HOW

Open `penguins.Rproj`, then run the scripts in
`analysis/` in order:

1. `01-clean-data.R`
2. `02-fit-model.R`
3. `03-make-figures.R`
```

[2] I swear I heard it from someone else, but I cannot find the source

```
Figures are written to `output/figures/`.

## Data

`data/penguins.csv` is from the {palmerpenguins}
package, saved on 2026-03-14. Do not edit this file.
```

The arrows are not part of the file. They are there to show you that a short README is not a summary of the project - it is six answers, in whatever order suits you.

If you are staring at an empty `README.md` and do not know where to start, write the six words down the page and fill them in.

Write the README first, if you can, and return to it often.

You can have **more than one README**. A short one inside `data-raw/` explaining where those files came from is often the most valuable file in the whole project.

And it's worth recording **when you obtained the data** - just noting the data, and also **what versions of software you used**, so that someone later, including you, can reconstruct similar conditions.

You can get the dependencies in an R project with:

```
renv::dependencies("path/to/directory")
```

Your Turn

Write a README for the project you sketched above. Answer the questions. Start it with:

```
usethis::use_readme_md()
```

Do it now, before you write any code - you will find it clarifies what you are actually about to do.

This pays off when the project becomes a paper

Everything in this lesson is framed around your future self. But the same structure is what makes your work usable by *anyone* else, and that matters most at publication.
Karthik Ram and I wrote about this in Common sense approaches to sharing tabular data alongside publication (*Patterns*, 2021). The argument, in short, is that depositing data doesn't make it reusable. The reusable part is mundane and structural:

- A README that says who, what, when, where and how.
- **Both** the raw data and the analysis-ready data, each in its own folder.

- The cleaning script that turns one into the other, kept with the raw data.
- Metadata describing each variable: its name, what it means, and its type. Enough to stop someone reading a gene sequence as a date.
- A licence. We recommend CC0.

If you organise your project this way from the start, you don't have to do anything special at submission. You're already most of the way there.
That's the real payoff, and it's why I'd rather you set this up now than tidy it up later.

1.9 Pitfalls of organisation

There are so many ways to organise a project. While there isn't a single **right** way, there are definitely **better** and **worse** ways. A key takeaway from today I want you to remember is:

> Your project might not be perfectly organised, but it should be possible for someone to understand how it is structured, and what to run first.

There are a few patterns that make picking up a new project hard. I am guilty of all of these. I am not judging you if these are how you work now. What I am saying is there are some ways you can improve.

You can see these patterns in the course exercises you downloaded earlier: https://github.com/njtierney/rbp-exercises

We will quickly move through these now.

1.9.1 Everything in one folder

```
01-everything-in-one-folder/
├── Rplot.png
├── Screen Shot 2026-03-14 at 10.23.45 am.png
├── Untitled1.R
├── analysis.R
├── figure1.png
├── notes.txt
├── penguins.csv
├── plot_stuff.R
├── report draft.docx
└── results.csv
```

Notice the R scripts, the data, the figures, a Word document, a screenshot, and `Untitled1.R`, are all *flat* - they are in the same folder. This works until there are more than about fifteen files, at which point finding anything becomes a search problem.

I think about this as like a grocery list: If my shopping list is only 5 items long, it might not impact me when I go to the shops:

- Milk
- Yoghurt
- Muesli
- Bananas
- Muesli bars

But if I'm buying a bunch of food for the week, some meal prep, and a few other bits and pieces, if I don't order the shopping list, and group the similar parts together, I am creating work for myself.

The two shopping lists

1.10 Unordered

- Milk
- Onion powder
- Red capsicum
- Yoghurt
- Muesli
- Bananas
- Muesli bars
- Tomato paste
- Ginger
- Coriander
- Garlic
- Lemongrass
- Coconut milk
- Chicken thighs
- Limes
- Eggplants
- Snow peas
- Thai basil
- 500g beef mince
- Basmati rice

1.11 Ordered

- **Produce**
  - Bananas
  - Ginger
  - Garlic
  - Lemongrass
  - Limes
  - Thai basil
  - Coriander
  - Eggplants
  - Red capsicum
  - Snow peas

- **Dairy**
  - Milk
  - Yoghurt
- **Meat**
  - Chicken thighs
  - 500g beef mince
- **Pantry**
  - Muesli
  - Muesli bars
  - Basmati rice
  - Coconut milk
  - Tomato paste
  - Onion powder

In this instance, I would recommend grouping the files into folders

1.12 Problem

Flat directory structure can make it hard to group things to understand how they are related.

```
01-everything-in-one-folder/
├── Rplot.png
├── Screen Shot 2026-03-14 at 10.23.45 am.png
├── Untitled1.R
├── analysis.R
├── figure1.png
├── notes.txt
├── penguins.csv
├── plot_stuff.R
├── report draft.docx
└── results.csv
```

1.13 Better

Put similar things into folders (directories) that give them structure.

```
01-everything-in-one-folder/
├── internals/
│   └── notes.txt
├── plots/
│   ├── Rplot.png
│   ├── Screen Shot 2026-03-14 at 10.23.45 am.png
│   └── figure1.png
```

```
├── scripts/
│   ├── Untitled1.R
│   ├── analysis.R
│   └── plot_stuff.R
├── data/
│   └── penguins.csv
└── outputs/
    ├── report draft.docx
    └── results.csv
```

1.13.1 No clear entry point

```
02-no-clear-entry-point/
├── analysis.R
├── clean.R
├── data.csv
├── do_it.R
├── final.R
├── helpers.R
├── main.R
├── model.R
├── plots.R
├── run.R
├── script.R
└── tests.R
```

There are eleven `.R` files and no way to tell which one runs first. If the only way to know the order they are run in, is only in your head, the project is not reproducible. It is a performance that only you can give.

1.14 Problem

When task order is required, but it is not written down, you cannot reproduce the task.

showing the above file preview

```
02-no-clear-entry-point/
├── analysis.R
├── clean.R
├── data.csv
├── do_it.R
├── final.R
├── helpers.R
├── main.R
├── model.R
├── plots.R
├── run.R
```

```
├── script.R
└── tests.R
```

1.15 Improvement

Encode the task order in another file, such as "00-run-first.R" - adding numbers to the start of the file will mean they sort the files in order. If files aren't useful anymore, you can delete them, or you can put them in an "attic" folder, if you aren't ready to let them go.

```
02-no-clear-entry-point/
├── 00-run-first.R
├── 01-helpers.R
├── 02-clean.R
├── 03-analysis.R
├── 04-model.R
├── 05-plots.R
├── 06-tests.R
├── 07-final.R
└── data/
    └── data.csv
└── attic/
    ├── xx-main.R
    ├── xx-do_it.R
    └── xx-script.R
```

1.15.1 File names encode history, not content

```
04-names-encode-history/
├── analysis.R
├── analysis2.R
├── analysis_final.R
├── analysis_final_USE_THIS_ONE.R
├── analysis_final_USE_THIS_ONE_v2 (nick edits).R
├── analysis_final_USE_THIS_ONE_v2.R
├── analysis_final_v2.R
└── penguins.csv
```

The *names* of these files tell a story: the analysis progressed, things changed, and there were concerns about knowing which version worked, and being able to go back and change things. You can think of this as using "track changes" in a word document. This is a kind of version control. But naming the files in order to track versions creates problems, and it is better solved with specific version control software, such as git. I cover this in another course, "Introduction to git and github".

1.16 Problem

There are multiples of the same file with names that tell you their version, rather than their task.

showing the above file preview

```
04-names-encode-history/
├── analysis.R
├── analysis2.R
├── analysis_final.R
├── analysis_final_USE_THIS_ONE.R
├── analysis_final_USE_THIS_ONE_v2 (nick edits).R
├── analysis_final_USE_THIS_ONE_v2.R
├── analysis_final_v2.R
└── penguins.csv
```

1.17 Solution

Name the files for what they do, not when you made them[3]. If you aren't ready to let them go, use the "attic" folder.

```
04-names-encode-history/
├── analysis.R
└── data/
    └── penguins.csv
└── attic
    ├── analysis2.R
    ├── analysis_final.R
    ├── analysis_final_USE_THIS_ONE.R
    ├── analysis_final_USE_THIS_ONE_v2 (nick edits).R
    ├── analysis_final_USE_THIS_ONE_v2.R
    └── analysis_final_v2.R
```

1.17.1 Editing raw data in place

You might have some data provided for analysis and there might be instructions to go through by hand to fix typos, or add new data. You should not make hand edits to the file, as this changes the history and increases the chances of a non-repeatable change. You can make a critical error by accidentally hitting the wrong key. From experience, having accidentally sorted a single column in an excel sheet and not realising this, I essentially turned a meaningful data source into random noise.

- The problem: Hand editing raw data

[3]Ideally, you would use a version control system like git. Out of scope for this course.

- The solution: Raw data should be read-only, and changes that happen to it happen with code, and are saved to separate outputs.

The way you actually enforce that is with two folders. `data-raw/` holds the file exactly as it arrived and nothing ever writes to it. `data/` holds the version your analysis reads, and it is produced by a script.

1.18 Problem

One file, edited by hand, and no way to tell what you changed or when.

```
04-editing-raw-data/
├── analysis.R
└── penguin-survey.xlsx     ← opened in Excel, typos fixed by hand
```

1.19 Better

```
04-editing-raw-data/
├── data-raw/
│   ├── penguin-survey.xlsx  ← exactly as it arrived. Read only.
│   ├── clean-penguins.R     ← every fix you would have made by hand
│   └── README.md            ← where it came from, and when
├── data/
│   └── penguins.csv         ← written by clean-penguins.R
└── analysis.R               ← reads data/, never the xlsx
```

Every correction is now a line of code. You can read it, re-run it, and disagree with it in six months. The typo fix that used to be a keystroke nobody saw is now `mutate(species = if_else(species == "Adelei", "Adelie", species))`, sitting in a script with the raw file beside it.

And it survives the thing that hand editing cannot: the data arriving again next year, slightly different. Re-run the script.

1.19.1 Outputs mixed in with inputs

```
03-outputs-mixed-with-inputs/
├── 01-clean.R
├── 02-model.R
├── cleaned_data.csv
├── fig-mass-flipper.png
├── fig-species.png
├── model_fit.rds
├── penguins.csv
└── raw survey data.xlsx
```

In this case, figures, model outputs, and cleaned data sit in the same folder as your raw data and scripts. You cannot tell at a glance what is necessary for your analysis, and what is produced as output by your analysis. This means you don't have a clear sense of how to reproduce files, or where edits can be safely made. A good test: *could I delete this folder entirely and regenerate it by running my code?* If yes, it is an output, and it should live somewhere that says so.

- The problem: Not knowing what is inputs vs outputs impacts reproducibility.
- The solution: Clearly label inputs and outputs - potentially put your outputs in an "outputs" folder. Or have a file that describes the inputs and outputs, such as a README, or an analysis script.

> **Your Turn**
>
> Open a project you worked on more than six months ago.
>
> 1. Can you tell, within thirty seconds, which script to run first?
> 2. Which files could you delete and regenerate from code?
> 3. Which files could you *not* regenerate? Those are the ones that actually matter - are they backed up?

1.20 "Good enough" project organisation

There is no single correct structure, but there is a common shape that I think gives you a great starting place:

```
my-project/
|-- my-project.Rproj
|-- README.md
|-- data-raw/
|   |-- penguins-raw.csv
|   `-- clean-penguins.R
|   `-- README.md
|-- data/
|   `-- penguins.csv
|-- R/
|   `-- functions.R
|-- analysis/
|   |-- 01-clean-data.R
|   |-- 02-fit-model.R
|   `-- 03-make-figures.R
`-- output/
    |-- figures/
    `-- models/
```

- **README** - as discussed at the start - this guides the reader through just what this is.

- **`data-raw/` holds the raw data, exactly as you received it**, along with the script that cleans it. Raw data is read-only, so nothing ever writes here, and you never edit it by hand. Keeping the cleaning script next to the raw data means the two never drift apart. Note that there is a README.md in data-raw/ to tell you a bit more about the data. You can have more than one README.
- **`data/` holds the analysis-ready data** that the cleaning script produces. This is the version your analysis actually reads.
- **`R/` holds functions.** Code that gets *used*, not code that gets *run*. We will spend a whole lesson on this.
- **`analysis/` holds scripts that get run**, in order. The numbering is doing real work: it tells your future self the entry point without needing a README.
- **`output/` holds things you can delete.** Anything in here can be regenerated by re-running the analysis.

If your project is small, collapse this. Three files in a flat folder with a README is a perfectly good project structure. You are not aiming for folders. You are aiming for *a place where each kind of thing goes*, and for being consistent about it.

> **Your Turn**
>
> Sketch the folder structure for a project you are working on right now. Don't create it yet - just write it out.
>
> 1. Where does raw data go?
> 2. Where do the things you can regenerate go?
> 3. Where does someone start reading?

1.21 How to name files

Naming is hard. We will come back to this throughout the course.

Having this "good enough" folder structure tells you **where** things go, but it does not tell you what to **name** them. So let's talk about that.

Some especially good pieces of advice:

- **Machine readable.** No spaces, no punctuation beyond `-` and `_`, and stick to one case. `01-clean-data.R`, not `01 Clean Data (final).R`.
- **Human readable.** The name should say what the file does. `clean-penguins.R` tells you something. `script2.R` does not.
- **Sorts sensibly.** Numbers at the front, zero padded, so `01`, `02`, `10` stay in order. Dates as `YYYY-MM-DD`, which sorts correctly for free.

That last one is the sneaky one. If your file names sort properly, your file explorer does your project organisation for you, and you get the running order without needing a README.

If you want to read more about this, I suggest:

- The files chapter of the Tidyverse style guide. It is short, and it is worth your time reading.
- Jenny Bryan's talk, "How to name files like a normie"

> **Your Turn**
>
> Look at the file names in your most recent project.
>
> 1. How many have spaces in them?
> 2. If you sorted them alphabetically, would the order mean anything?
> 3. Pick the worst named file and give it a better name.

1.22 Make your project "good enough"

I want to be clear that the goal here is *good enough*, not perfect. This phrasing comes from a paper I recommend to everyone, "Good enough practices in scientific computing" by Wilson et al.

The principles I would hold you to:

1. **Put each project in its own folder, with its own `.Rproj` file.**
2. **Treat raw data as read-only.** Never edit it by hand.
3. **Treat generated output as disposable.** If you can't delete it and get it back, it isn't really output.
4. **Use relative paths.** Never `setwd()`.
5. **Name files so a stranger can guess what they do.**
6. **Make the running order obvious**, with numbered scripts or a README.
7. **Write a README.**

That is the whole list. If you do those seven things, you are organised enough, and the remaining effort is better spent on the analysis itself.

One more thing:

> Don't reorganise everything tonight.

Apply this to your *next* project. Retrofitting a large existing project is a big job, and can be a very effective way to talk yourself out of the whole idea. Start clean, once, and let the habit build.

> **Your Turn**
>
> Go through the seven principles above and score your most recent project out of seven.
> Pick the single one that would have saved you the most time, and write down what you would have to change to fix it.

Summary

- You are always collaborating with your future self.
- A file path is the address of a file; absolute paths only work on one machine.
- Use one folder and one `.Rproj` per project, and relative paths within it.
- `here::here()` builds paths from the project root, wherever the code is called from.
- Raw data is read-only; output is disposable.
- Make the running order obvious.
- Write a README that says what, how, and where from.
- Start R with a blank slate, and restart often.

Next, we will look at what happens *inside* those files: how code is styled, and why consistency makes code readable.

2 Code style and readability

> Good coding style is like correct punctuation: you can manage without it, butitsuremakesthingseasiertoread.
>
> – Hadley Wickham:

There is more to your code than having it run. We are aiming for your code to be "running" AND "easy to read, and easy to understand".

There are some really useful tools we can apply to get quick wins on improving the readability of your code. In this section we will discuss concepts of **code style**, talking about how to easily improve code **layout (whitespace, newlines, punctuation standards)**, and improving code with **code linting** - where a program looks at how your code is written and identifies common errors, style inconsistencies, and other known problem patterns.

Overview

Duration 45 minutes
Questions

- Why does style matter, if the code runs either way?
- What counts as style: layout, naming, or correctness?
- How should I lay out a pipeline, and what should I call things?
- What is a code formatter, and how do I get one to run on save?
- What is a linter, and what does it catch that a formatter does not?

2.1 Reading things that are hard to read is hard

Which of these is easier to read?

2.2 quote 1

> i dont understan an ingle werd sumtimes of wot i say, but it is becoz i am so cleva

– Oscar Wilde, The Happy Prince and Other Stories

2.3 quote 2

> I am so clever that sometimes I don't understand a single word of what I am saying

– Oscar Wilde, The Happy Prince and Other Stories

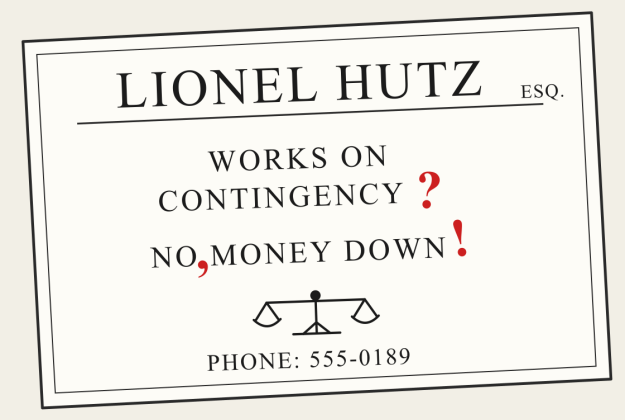
One of these is harder to read than the other. It is easier to read the text that has correct spelling and grammar.

Punctuation is an example of grammar - it plays a role in deciding what the words mean.

Here is Lionel Hutz, from The Simpsons, attorney at law.

2.4 The card

2.5 Marked up



The punctuation changed the meaning!

You can still work out what it says. But you spend energy doing it. Our precious mental energy.

We want our code to be easier to understand, when we're reading a big lump of text:

2.6 As it arrived

> we ran the model on the new data and it's results where different to what we expected. the team thought maybe the cleaning script had changed But after checking, changed it had not. Their was one column read in as character not a number, which meant the mean was computed over text and returned NA this is the kind of thing thats easy to miss when your reading quickly

2.7 Tidied up

> We ran the model on the new data, and its results were different to what we expected. The team thought maybe the cleaning script had changed, but after checking, it had not. There was one column read in as a character rather than a number, which meant the mean was computed over text and returned `NA`. This is the kind of thing that is easy to miss when you are reading quickly.

Both say the same thing. The second one you read once.

Going through the first, you were doing unpaid work the whole way - and these little bumps interrupt your thinking:

- Silently correcting "their" to "there".
- Has a sentence ended? There was a capital letter.
- Re-reading the sentence, it sounds backwards.

Our energy and our attention are finite resources, and if we spend them on these smaller things, then we will spend less energy on the goal: understanding the meaning.

2.7.1 CPU cycles vs mental cycles

A computer has CPU cycles, and we spend real effort saving them - avoiding a copy of a big object, shaving milliseconds off something that runs a thousand times.

You have **mental cycles** too, and yours are far more expensive.

So: terse code saves the machine a tenth of a second, and then you spend an extra minute reading it every time you come back. Nothing got faster. The work moved off the machine, which has cycles to spare, and onto you, who does not.

Spend the computer's freely. Guard your own.

Code is the same, and R has its own version of the Hutz card:

```
x <- 1
x < -1
```

```
[1] FALSE
```

The first line assigns `1` to `x`. The second asks whether `x` is less than negative one. One space, and a completely different meaning.

R accepted both without complaint, which is the whole problem.

Both of these take the chicks at day 21, and ask which diets came out above the average weight.

```
library(tidyverse)
d<-ChickWeight
M<-mean(d$weight)
d<-d▷filter(Time==21)
r<-d▷group_by(Diet)▷summarise(m=mean(weight),n=n())▷arrang
↪  e(desc(m))▷mutate(f=m>M)
r
```

```
# A tibble: 4 x 4
  Diet      m     n f
  <fct> <dbl> <int> <lgl>
1 3      270.    10 TRUE
2 4      239.     9 TRUE
3 2      215.    10 TRUE
4 1      178.    16 TRUE
```

```
library(tidyverse)

overall_mean <- mean(ChickWeight$weight)

chicks_day_21 <- ChickWeight ▷
  filter(Time == 21)

diet_summary <- chicks_day_21 ▷
  group_by(Diet) ▷
  summarise(
    weight_mean = mean(weight),
    n_chicks = n()
  ) ▷
  arrange(desc(weight_mean)) ▷
  mutate(above_overall_mean = weight_mean > overall_mean)

diet_summary
```

```
# A tibble: 4 x 4
  Diet  weight_mean n_chicks above_overall_mean
  <fct>       <dbl>    <int> <lgl>
1 3            270.       10 TRUE
2 4            239.        9 TRUE
3 2            215.       10 TRUE
4 1            178.       16 TRUE
```

These both run. R does not care where you put your spaces. R does not care that your variables are short, that your code is very long-winded. Or short-winded.

But you care about these things - every stray space, and every inconsistent name is one more small correction you make before you can get at what the code is trying to do.

You are spending your mental cycles on this problem. It would be nice if you didn't have to do that.

2.7.2 Both of those are wrong

Let's look closely at that last column. Every diet is above the overall mean. Weird? This happens because the overall mean is taken from `ChickWeight` **before** the filter. It is the average weight across every day of the study - day 0 chicks included, when they weighed about 40 grams. Every day 21 chick clears that easily.

```
mean(ChickWeight$weight) # every day
```

```
[1] 121.8183
```

```
mean(ChickWeight$weight[ChickWeight$Time == 21]) # day 21 only
```

```
[1] 218.6889
```

Both versions have the bug. Good layout doesn't fix the bug, but it makes it (in my opinion!) easier to notice. `M` and `m` tell you nothing, so there is nothing to be suspicious about. `overall_mean` and `weight_mean` sitting on the same line, makes us ask: the overall mean of *what*?

The style helps buy back some of your attention.

And that is part of what this lesson is about - there are ways to automate a lot of this! And there are ways to turn the rest into a habit, so it stops being a decision you make every time you write a line.

Nearly all of that surface level noise is systematic, which means we can get rid of it systematically.

And once it's gone, your attention goes to the part that actually needs you: whether the code does the right thing.

2.8 Style is like grammar

Style guides define a set of rules that keep code easy to read. To quote Hadley Wickham:

> Good coding style is like correct punctuation: you can manage without it, butitsuremakesthingseasiertoread.

2.8.1 Style: layout, naming, correctness

It helps to split "style" into two parts, because they need very different things from you.

Layout. Where the spaces go, where the line breaks go, how deep the indentation is, how a long function call gets split across lines.

```
# layout
mean(x,na.rm=TRUE)
mean(x, na.rm = TRUE)
```

This part is completely mechanical, and a tool can do all of it.

Naming. What you call your variables, your functions, and your files.

```
# naming
d2 <- d[d$Time == 21, ]
chicks_day_21 <- ChickWeight[ChickWeight$Time == 21, ]
```

This one is yours, because a name is a decision about meaning.

There is a third thing that is not style at all, but often gets lumped in with it.

Correctness. Whether your code actually does what you think it does.

This one is worth doing slowly, with two examples, because the failures are quiet.

Comparing two numbers. You would think this is safe:

```
0.1 + 0.2 == 0.3
```

```
[1] FALSE
```

No! Those numbers are stored as binary approximations and the answer is a hair off:

```
0.1 + 0.2 - 0.3
```

```
[1] 5.551115e-17
```

So you reach for `all.equal()`, which allows a small tolerance:

```
all.equal(0.1 + 0.2, 0.3)
```

```
[1] TRUE
```

Good. Now watch what it does when the two things are genuinely different:

```
all.equal(1, 2)
```

```
[1] "Mean relative difference: 1"
```

It does not return `FALSE`. It returns **a sentence**. And `if()` cannot do anything with a sentence:

```
if (all.equal(1, 2)) "same" else "different"
```

```
Error in `if (all.equal(1, 2)) ...`:
! argument is not interpretable as logical
```

The fix is `isTRUE()`, which asks the narrower question "is this exactly `TRUE`":

```
if (isTRUE(all.equal(1, 2))) "same" else "different"
```

```
[1] "different"
```

What that means in an analysis. While the two values agree, `all.equal()` returns `TRUE`, your `if` works, and you never learn there is a problem. It only breaks on the day the values differ - which is precisely the day you wrote the check for.

Recoding a factor. Here is one that does not error at all:

```
grade <- factor(c("low", "high", "low"))
ifelse(grade == "low", grade, "other")
```

```
[1] "2"     "other" "2"
```

You asked for `"low"` and got `"2"`. A factor is stored as integers with labels attached:

```
as.integer(grade)
```

```
[1] 2 1 2
```

```
levels(grade)
```

```
[1] "high" "low"
```

`ifelse()` drops the labels and hands back the bare codes. So `"low"` became `"2"`, its position in the alphabetical list of levels.

What that means in an analysis. Nothing warns you. You now have a column where a category has been replaced by its alphabetical rank, and every join, plot and table built on it downstream is quietly wrong. Add a new level next year and the numbers all shift.

`dplyr::if_else()` is stricter about types and keeps the factor:

```
dplyr::if_else(grade == "low", grade, factor("other"))
```

```
[1] low   other low
Levels: high low other
```

Both of those problems are laid out perfectly well. Every space is in the right place. A formatter has nothing to say about either of them, and neither does a careful read, most days. That is a *linter's* job, and we come back to it later in this chapter.

So.

- Layout is automatable
- Naming is yours
- Correctness is a different tool

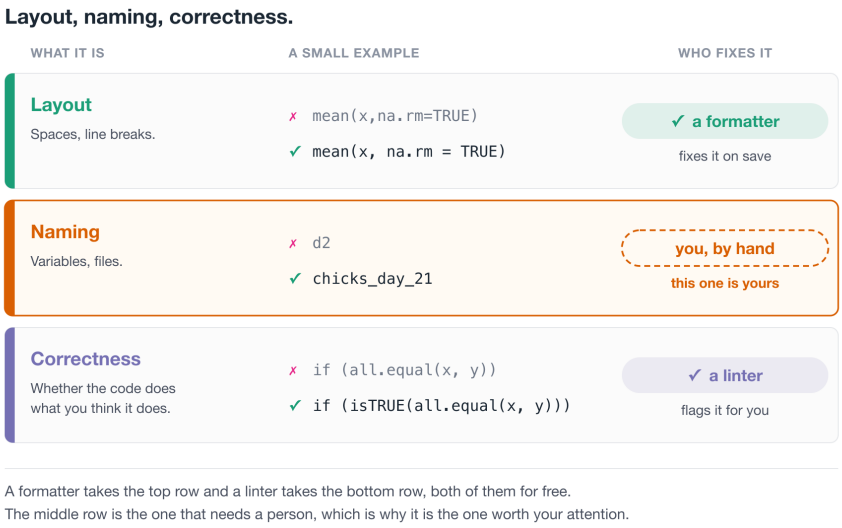


Figure 2.1: A formatter takes the top row and a linter takes the bottom row. The naming in the middle is the row that needs a person.

Let's get into it.

2.9 Layout

Lines are free. Use them.

> **Your Turn: a quick check on pipes**
>
> Everything from here on uses the pipe, ▷, so let's make sure we are all in the same place before we start.
>
> - Have you used ▷, or `%>%`, before?
> - Could you say out loud what this does?
>
> ```
> penguins ▷ filter(species == "Adelie")
> ```
>
> I like to read the pipe as **then**:
>
> > Take the penguins data, **then** filter it to the rows where species is "Adelie".
>
> The pipe gets a proper treatment in Workflow: code style in *R for Data Science*. It's a great book!

Let's look at the same code twice.

2.10 Hard work

```
penguin_summary<-penguins▷filter(!is.na(bill_len))▷group_by
↪ (species,island)▷
summarise(bill_mean=mean(bill_len),bill_sd=sd(bill_len),n=n())
```

2.11 Easy work

```
penguin_summary <- penguins ▷
  filter(!is.na(bill_len)) ▷
  group_by(species, island) ▷
  summarise(
    bill_mean = mean(bill_len),
    bill_sd = sd(bill_len),
    n = n()
  )
```

2.12 No pipe at all

```
penguin_summary <- summarise(
  group_by(
    filter(penguins, !is.na(bill_len)),
    species,
    island
  ),
```

```
  bill_mean = mean(bill_len),
  bill_sd = sd(bill_len),
  n = n()
)
```

All three give the same answer. R sees no difference at all.

But in the second one you can see the shape of the analysis:

- filter()
- group_by()
- summarise()

Three steps, one per line, each indented to show it belongs to the pipeline.

In the first one you have to parse it yourself, character by character, before you can even start thinking about whether the analysis is right.

The third one is the interesting case, because it is the one that looks fine. Every space is in the right place, the indentation is correct, and a formatter would leave it exactly as it is.

It is still harder to read, because the steps now run inside out. `filter()` happens first and sits deepest. `summarise()` happens last and sits at the top. To read it you have to find the innermost bracket, then work your way out, holding each layer in your head as you go. To read the pipe version you start at the top and go down.

That is a different problem from layout, and no formatter will fix it for you. It is also where rainbow parentheses start earning their keep - once you are three or four brackets deep, matching them by eye becomes its own small job, on top of the actual work.

Your Turn

Take this and work out, by reading alone, what it produces:

```
x<-lm(body_mass~flipper_len+species,data=penguins[pengui
↪  ns$year==2008&!is.na(penguins$sex),])
```

1. How many things are being filtered out?
2. What is the response variable?
3. How long did that take you?
4. How would you rewrite this?

2.12.1 The one line special

I once saw someone writing code where the goal, as far as I could tell, was to have as few lines as possible.

Not fewer *steps*. Fewer *lines*.

It was an entire analysis. Read the data in, reshape it, fit a model, draw a plot, save it. Four lines.

Have a look at both of these, and see which one you would rather open on a Monday morning.

2.13 Four lines

```
penguins <- read_csv("data/penguins.csv") ▷
 ↪  filter(!is.na(bill_length_mm), !is.na(body_mass_g)) ▷
 ↪  mutate(mass_kg = body_mass_g / 1000, bill_ratio =
 ↪  bill_length_mm / bill_depth_mm) ▷ group_by(species) ▷
 ↪  mutate(mass_z = (mass_kg - mean(mass_kg)) / sd(mass_kg))
 ↪  ▷ ungroup()
fit <- lm(mass_kg ~ bill_ratio + species + island, data =
 ↪  penguins)
p <- ggplot(penguins, aes(x = bill_ratio, y = mass_kg, colour
 ↪  = species)) + geom_point(alpha = 0.6) + geom_smooth(method
 ↪  = "lm", se = FALSE) + facet_wrap(~island) + labs(x = "Bill
 ↪  length / bill depth", y = "Body mass (kg)") +
 ↪  theme_minimal(base_size = 12)
ggsave("output/figures/mass-bill-ratio.png", p, width = 8,
 ↪  height = 5, dpi = 300)
```

2.14 The same thing, laid out

```
penguins <- read_csv("data/penguins.csv") ▷
  filter(
    !is.na(bill_length_mm),
    !is.na(body_mass_g)
  ) ▷
  mutate(
    mass_kg = body_mass_g / 1000,
    bill_ratio = bill_length_mm / bill_depth_mm
  ) ▷
  group_by(species) ▷
  mutate(mass_z = (mass_kg - mean(mass_kg)) / sd(mass_kg)) ▷
  ungroup()

fit <- lm(mass_kg ~ bill_ratio + species + island, data =
 ↪  penguins)

p <- ggplot(penguins, aes(x = bill_ratio, y = mass_kg, colour
 ↪  = species)) +
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE) +
  facet_wrap(~island) +
  labs(
```

```
    x = "Bill length / bill depth",
    y = "Body mass (kg)"
  ) +
  theme_minimal(base_size = 12)

ggsave(
  "output/figures/mass-bill-ratio.png",
  p,
  width = 8,
  height = 5,
  dpi = 300
)
```

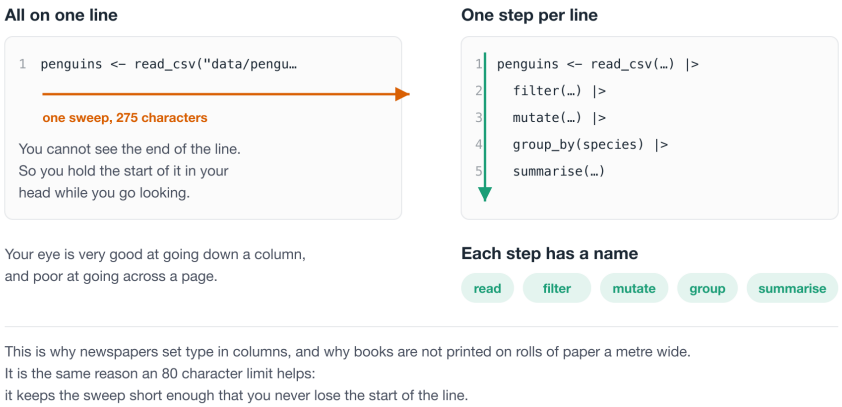


Figure 2.2: Why code reads better down a column than across a page.

Same code. Same results.

But look at what your eye can do with the second one. You can run down the left hand edge and read the steps off like a list: filter, mutate, group by, mutate, ungroup.

Your eye is very good at scanning down a column. It is terrible at scanning across a page, which is why newspapers use columns and why books are not printed on rolls of paper a metre wide.

The first version makes you do the horizontal thing. On a laptop you cannot even see the end of those lines without scrolling sideways, so you are holding the start of the line in your head while you go looking for the end of it.

Those are mental cycles, and you are burning them on the shape of the code rather than on the analysis.

There is a second thing that shows up the moment something breaks. R tells you which line the error is on. In the first version that is not much help, because a quarter of your analysis is on that line.

Lines are free. Use them.

2.15 Naming

This is the half of style that a formatter cannot help you with.

Pick a convention, `snake_case`, `camelCase`, or `CamelCase`, and stick to it. I prefer `snake_case` because I find it easier to read, but the choice matters far less than the consistency.

Consistency matters for a reason more concrete than tidiness. Compare:

```
weight_mean <- mean(ChickWeight$weight)
sd_weight <- sd(ChickWeight$weight)
```

with:

```
weight_mean <- mean(ChickWeight$weight)
weight_sd <- sd(ChickWeight$weight)
```

The first pair changes the order of the naming. The second sticks to one pattern, `variable_operation`, where the first word says what we measured and the second says what we did to it.

Why does that help? **Tab complete.**

If I am consistent, I can type `weight_` and press tab, and R tells me every single thing I have done to `weight`.

If I had chosen the other order, `sd_` would tell me everything I have taken a standard deviation of:

```
sd_weight <- sd(ChickWeight$weight)
sd_time <- sd(ChickWeight$Time)
```

Either order is fine. What you can't do is mix them.

Because then you have to *remember* which way round you named each thing, and that's exactly the kind of small tax that makes code tiring to work with.

2.16 Inconsistent

```
sd_Weight
SD_weight
Long_variable_Name
timeSD
```

2.17 Consistent

```
weight_mean
weight_sd
time_mean
time_sd
```

2.18 Read a style guide, once

It's worth your time to read through a style guide once. Properly, start to finish, one time.

You'll notice things in your own code that you didn't see before, and you'll see other people's code differently. It's a bit like learning about kerning.

Once you see it, you notice it everywhere.

> **Your Turn**
>
> Read the style chapter of Advanced R, 1st edition. It is short, and you can get through it in about ten minutes.
> As you read, keep a list of the rules you are currently breaking. Don't fix anything yet.

> **Going deeper**
>
> When you have more time, read sections 1 to 5 of the Tidyverse style guide: files, syntax, functions, pipes, and ggplot2.
> It is the more complete treatment, and it explains the reasoning behind each rule rather than just stating it.
> The files chapter is worth reading alongside the project organisation lesson, because it is really about project structure rather than code.

2.19 Do I have to do all this by hand?

No. And you shouldn't.

If you try to apply a style guide by hand you will spend your time putting spaces after commas, lining up arguments, and re-indenting things you just moved. It is tedious, and you will do it inconsistently, because it is exactly the kind of task humans are bad at and get bored by.

Worse, you'll do it *instead of* the analysis.

The good news is that layout is mechanical, and mechanical things can be automated.

2.20 Using the {air} formatter

Air is an R formatter from Posit. You point it at your code, and it lays it out properly. It is basically magic.

Once you have it wired into your editor, you stop thinking about layout. You type whatever comes out of your head, hit save, and it comes out tidy. You don't need to spend another moment going over your code tweaking spaces.

2.20.1 Checking air is installed

You installed air in the setup chapter - Code formatters. We just need to check it is there, and find out where it lives.

> **Your Turn: check air is installed**
>
> 1. Open a terminal. In RStudio that's the Terminal tab, next to the Console.
> 2. Run `air --version` and check you get a number back.
> 3. Run `which air` on macOS or Linux, or `where air` on Windows, and **write the path down**. You need it in a minute.
>
> If you get "command not found", the install didn't finish - go back to Code formatters and run it again.
> If it won't install, don't spend the whole session fighting it. Use `{styler}` for today, and come back to air later.

2.20.2 If you can't install air, use {styler}

If you can't install {air} because of network issues, use `{styler}`. This is an R package that pre dates {air}. Install it with:

```
install.packages("styler")
```

It follows the same tidyverse style guide air does, so the output is very close.

There are a few differences between air and styler, basically: air is faster, and it is less configurable.

2.21 Format on save (aka magic)

This changes how you work!

Once this is set up, you write code however it falls out of your head. Wonky spacing, arguments strewn across the line, indentation all over the place. Do not fix any of it.

Hit `Cmd / Ctrl + S`, and it is all just correct.

I cannot really oversell this.

2.22 RStudio

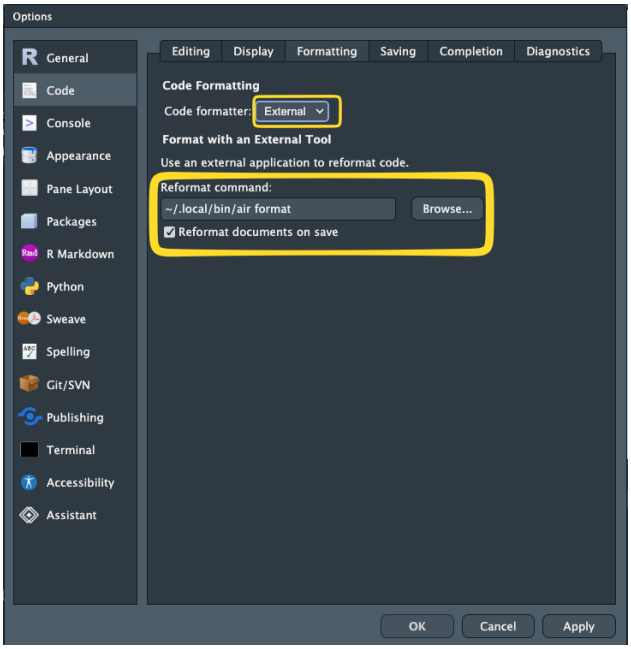
You need RStudio 2024.12.0 or later.

You want the path you wrote down a moment ago, from `which air` or `where air`.

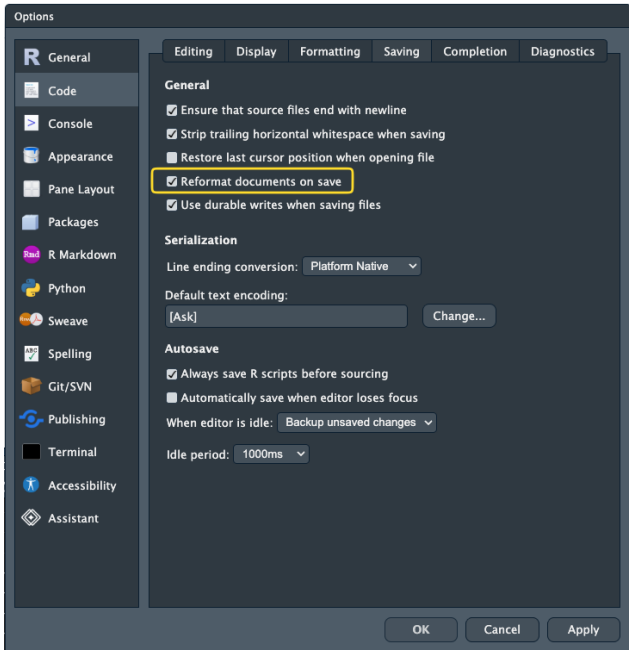
Go: Tools → Global Options → Code → Formatting.

- For {air} Set **Code formatter** to **External**.
- For {styler} Set **Code formatter** to **Styler**.

Set **Reformat command** to your air path followed by `format`, so something like `/Users/nick/.local/bin/air format`. If your path has spaces in it, wrap it in double quotes.



Now turn on format on save. Go: Tools → Global Options → Code → Saving, and tick **Reformat documents on save**.



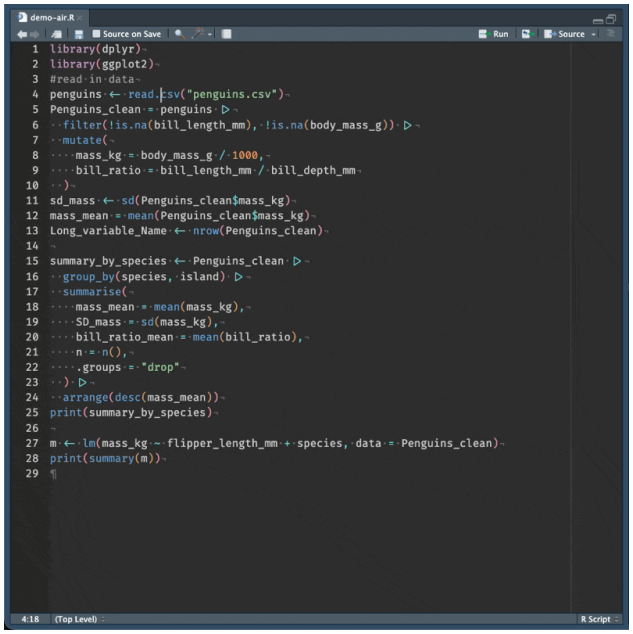


Figure 2.3: The file after saving. This one is a screen recording in the web version of the book.

2.23 Positron

Air comes with Positron, so there is nothing to install.

Open the command palette (`Cmd/Ctrl + Shift + P`) and run **Air: Initialize Workspace Folder**. This writes the recommended settings into your project.

Figure 2.4: What the command writes for you. This one is a screen recording in the web version of the book.

If you'd rather do it by hand, add this to your `settings.json`:

```
{
  "[r]": {
    "editor.formatOnSave": true,
    "editor.defaultFormatter": "Posit.air-vscode"
  }
}
```

For R chunks inside Quarto documents, add the Quarto formatter too:

```
{
  "[quarto]": {
    "editor.formatOnSave": true,
    "editor.defaultFormatter": "quarto.quarto"
  }
}
```

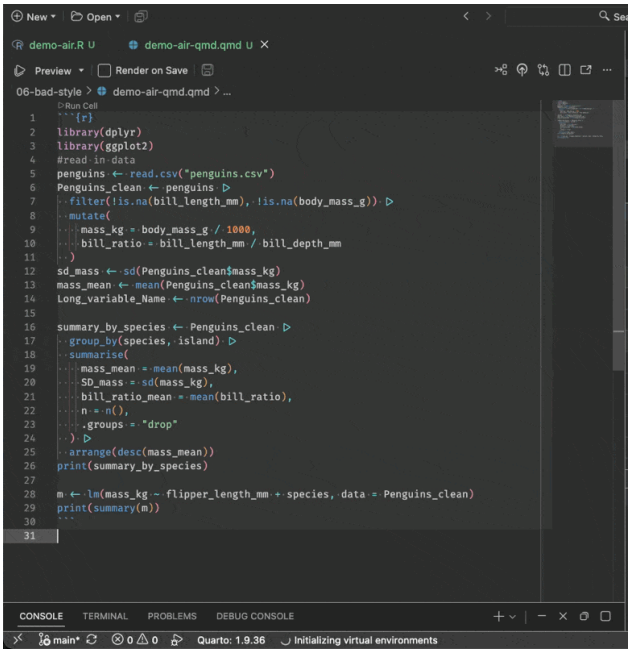


Figure 2.5: The R chunk inside a Quarto document, after saving. This one is a screen recording in the web version of the book.

Setting it up from R

If you're inside a project already, this does most of the work for you:

```
usethis::use_air()
```

It sets up the editor configuration and adds an `air.toml` to your project, which gives you some options to customise air.
This isn't mandatory, but can be useful.

Your Turn: the good bit

1. Set up format on save in your editor, using the tabs above.
2. Open any R script. Make a mess of it on purpose. Delete the spaces around a ←, jam some arguments together, break the indentation.
3. Hit `Cmd / Ctrl + S`.
4. Do that three or four more times, with a different kind of mess each time,

I want you to do it repeatedly, because you don't have to tidy your own code, and that's awesome.

2.23.1 Formatting a file, or a whole project

You don't need format on save, or even an editor. Both tools will format one file, or everything at once.

2.24 air

Air will format a single file:

```
air format analysis/01-clean-data.R
```

or an entire project:

```
air format .
```

That second one is the command I run most. It is also the one worth putting in front of any code review, so that nobody spends their attention on spacing.

2.25 styler

Styler does the same two jobs from inside R, so there is no terminal involved:

```
styler::style_file("analysis/01-clean-data.R")
styler::style_dir(".")
```

Your Turn

There is a deliberately awful script in the course repository at `06-bad-style/messy-analysis.R`. It breaks essentially every rule in this chapter.

1. Open `messy-analysis.R` and read it. Note how long it takes you to work out what it does.
2. Format it with air or styler.
3. Read it again. What can you see now that you couldn't before?
4. Look at the variable names in the formatted version. Air has not touched a single one of them, and they're still a mess. Write down three you would rename.

Notes on using air

A few things that will save you some confusion.
There is very little to configure, and that's on purpose. Air's settings live in an `air.toml` file, and there are about ten of them. The ones you might actually change are `line-width` (default 80) and `indent-width` (default 2). There is no way to spend an afternoon tuning it, which is exactly why I like it.
It won't work on an untitled tab. Air needs a file that exists on disk, with an `.R` extension, so that it knows it's looking at R code. Save the file first.
In RStudio it won't work inside R chunks in Quarto or R Markdown. This is an RStudio limitation rather than an air one, and it may well change. Positron handles chunks fine.
If your code doesn't reformat, you probably have a syntax error. Air will not format code it cannot parse. So a file that stubbornly refuses to tidy up is telling you something useful. Go and find your missing bracket.
That last one turns out to be a small free bonus. Format on save quietly becomes a syntax check on save.

2.26 Using linters

Air has now taken care of how your code *looks*. Here is a piece of code where that isn't the problem.

```
sites <- character(0)

for (i in 1:length(sites)) {
  message("Processing site ", sites[i])
}
```

Air is perfectly happy with that. Every space is in the right place, the indentation is correct, and running `air format` on it changes nothing

at all.

It is also broken.

When `sites` is empty, `length(sites)` is `0`, and `1:0` does not give you nothing. It gives you this:

```
sites <- character(0)
1:length(sites)
```

```
[1] 1 0
```

So the loop runs twice, on elements `1` and `0`, neither of which exists. No error, no warning, and a message about a site that isn't there.

This is a different kind of problem, and it needs a different kind of tool.

- A **formatter** changes how your code *looks*. `{air}` doesn't care what your code does.
- A **linter** tells you what your code might be doing *wrong*. It reads your code without running it, which is called static analysis, and flags errors, bugs, and suspicious patterns.

You want both. Formatting settles arguments about layout. Linting catches the class of mistake that runs perfectly happily and gives you the wrong answer.

> Some history: why is it called a linter?
>
> The name comes from a program called `lint`, written by Stephen C. Johnson at Bell Labs in 1978 to check C code for bugs and portability problems. You can read more on the Wikipedia page for lint.
> Lint is the fluff that clothes shed in the wash, and a clothes dryer has a lint trap to catch it. Johnson's idea was that his program would work the same way, catching the waste fibres while leaving the fabric intact.
> Your code is the fabric. The linter is the trap.
> I like this because it sets the right expectation. A lint trap does not tell you the jumper was a bad idea. It picks off the fluff, and it does it every single load, without being asked.

2.26.1 jarl

Jarl, which stands for Just Another R Linter, is a fast linter by Etienne Bacher. It is built on top of Air, which is why the two feel like the same tool.

Here is a script of the kind most of us have written. Nothing exotic. Read some data in, flag a few things, print a message.

```r
penguins <- read.csv("data/penguins.csv")

missing_mass <- penguins$body_mass_g == NA

penguins$heavy <- ifelse(penguins$body_mass_g > 5000, T, F)

has_data <- nrow(penguins) > 0

if (has_data == TRUE) {
  message("Loaded ", nrow(penguins), " penguins")
}
```

Air has no complaints. It runs without an error.

Point a linter at it, though, and you get three warnings. Use whichever of these you have:

2.27 jarl

```
$ jarl check penguin-check.R
```

2.28 flir

```r
flir::lint("penguin-check.R")
```

```
warning: equals_na
 --> penguin-check.R:3:17
  |
3 | missing_mass <- penguins$body_mass_g == NA
  |                 ------------------------- Comparing to NA with `==`,
  |
  = help: Use `is.na()` instead.

warning: true_false_symbol
 --> penguin-check.R:5:55
  |
5 | penguins$heavy <- ifelse(penguins$body_mass_g > 5000, T, F)
  |                                                       - `T` and `F` ca
  |

warning: redundant_equals
 --> penguin-check.R:9:5
  |
9 | if (has_data == TRUE) {
  |     ---------------- Using == on a logical vector is redundant.
  |
```

Let's take those one at a time, because each one teaches something.

`missing_mass ← penguins$body_mass_g == NA`

This is the one I want you to remember. It looks completely reasonable, and it is always wrong.

`NA` means "I don't know". So asking "is this value equal to a value I don't know?" can only be answered with "I don't know":

```
x <- c(1, NA, 3)
x == NA
```

```
[1] NA NA NA
```

Every answer is `NA`, including for the values that are obviously not missing. Your `missing_mass` vector contains no information at all! Note that jarl even told you want to do!

```
is.na(x)
```

```
[1] FALSE  TRUE FALSE
```

That's the function you wanted.

`ifelse( … , T, F)`

`TRUE` and `FALSE` are reserved words in R. `T` and `F` are just ordinary variables that happen to start out holding `TRUE` and `FALSE`, which means anyone can reassign them.

```
T <- FALSE
mean(c(1, NA, 3), na.rm = T)
```

```
[1] NA
```

```
rm(T)
```

`na.rm = T` quietly became `na.rm = FALSE`, and the mean is now `NA`. Spell them out and this cannot happen to you.

`if (has_data == TRUE)`

`has_data` is already `TRUE` or `FALSE`. Comparing it to `TRUE` gets you back exactly what you started with, so `if (has_data)` says the same thing with less to read.

This one is not a bug. It is a mental cycle.

! Two nastier ones, for later

These are much harder to spot by eye.
`class(dat) == "data.frame"` looks fine until you meet an object with more than one class:

```
class(data.frame(x = 1))
```

```
[1] "data.frame"
```

```
class(datasets::ChickWeight)
```

```
[1] "nfnGroupedData" "nfGroupedData"  "groupedData"    "data.frame"
```

One class for a plain data frame, four for that one. So the comparison hands `if()` four answers where it wanted one, and errors. `inherits(dat, "data.frame")` asks the question you actually meant.
`if (all.equal(x, y))` is worse. `all.equal()` returns `TRUE` when things match, and a *description of the difference* when they don't:

```
all.equal(1, 1)
```

```
[1] TRUE
```

```
all.equal(1, 1.1)
```

```
[1] "Mean relative difference: 0.1"
```

`if()` can't do anything sensible with it. So the code works perfectly while your values match, and falls over the first time they don't, which is precisely the case you wrote the check for. `isTRUE(all.equal(x, y))` fixes it. (demo).

Notice that these fail in two different ways.

`== NA` and `na.rm = T` are **silent**. They run, there is no error and no warning, and the answer is wrong.

That is what a linter is for. Careful reading **could** catch all of these. But a tool catches them **every** time.

Where a fix is clear and safe, jarl or flir can apply it for you:

2.29 jarl

```
$ jarl check penguin-check.R --fix
```

2.30 flir

```
flir::fix("penguin-check.R")
```

which turns the `T` and `F` into `TRUE` and `FALSE`. It won't touch the `== NA`, because deciding what you actually meant there is your job, not the tool's.

It ships 55+ rules, integrates with VS Code, Positron and Zed, and runs in CI. It's also very fast. On the `dplyr` source, roughly 25,000 lines of R, the documented benchmark is 0.131 seconds against 18.5 seconds for `{lintr}`.

> **What about lintr?**
>
> `{lintr}` is the long established R linter, it's pure R, and it's what you'll meet in most existing projects and CI setups. It's a perfectly good tool and knowing it is worthwhile.
> Jarl is newer and much faster, which matters more than it sounds. A linter you run on every save changes your habits.
> Use something! Either one of them.

2.30.1 If you can't install jarl, use {flir}

Same story as air. Jarl is a command line tool, so the install can go wrong in all the same ways.

Your backup is `{flir}`, which is an ordinary R package:

```
install.packages("flir")
```

It's by Etienne as well, and it's fast for the same sorts of reasons. The documented benchmark is 172 milliseconds against 3.44 seconds for `{lintr}` on the same code.

Two functions do the work:

```
flir::lint("06-bad-style/dodgy-logic.R")
flir::fix("06-bad-style/dodgy-logic.R")
```

`lint()` tells you what it found. `fix()` applies the changes it's confident about, the same way `jarl check --fix` does.

Call them with `flir::` in front, as above. If you run `library(flir)` instead, its `fix()` will mask the `fix()` that comes with R, and you'll get a warning.

One thing you should know. Etienne has moved on to jarl, and says on the flir site that he won't be adding new rules to it. So flir is finished rather than growing.

For our purposes that's fine. It catches everything in this chapter, it installs anywhere R installs, and it fixes what it can.

> **Your Turn**
>
> Back to `06-bad-style/`. There's a second file there, `dodgy-logic.R`, which is formatted perfectly well and still wrong.
>
> 1. Read it, and see if you can spot the bug by eye. Give yourself two minutes.
> 2. Run `jarl check 06-bad-style/dodgy-logic.R`. If you don't have jarl, use `flir::lint("06-bad-style/dodgy-logic.R")` or `lintr::lint("06-bad-style/dodgy-logic.R")`.
> 3. Now run `jarl check` over your own most recent project. What comes up?
>
> Nothing on that last one is a judgement. I run this over my own code and it finds things every time.

> **Read more**
>
> The style and naming material in this chapter is adapted from my blog post How to get good with R.

Summary

- Badly laid out code makes you spend mental cycles on the surface before you get to the ideas, in the same way that bad spelling does.
- Mental cycles are worth more than computer cycles. Spend the machine's, guard your own.
- Lines are free. Use them, because your eye reads down far better than across.
- Pick a naming convention and stick to it, because consistency is what makes tab complete work for you.
- Read a style guide once, properly, and you'll see code differently afterwards.
- Set up `{air}` and format on save, then stop thinking about layout.
- A linter is a different tool again, and it catches the bugs that run perfectly happily.

Next, we look at readability beyond layout: names that carry meaning, breaking a wall of code into pieces, and how to review code.

3 Writing readable code

> "Programs must be written for people to read, and only incidentally for machines to execute."
>
> – Harold Abelson and Gerald Jay Sussman, Structure and Interpretation of Computer Programs

So far we have focussed on structural things I would class as "easy wins":

- File paths
- Reproducibility fixes
- Quality of life improvements to your editor
- Creating projects
- Project structure (Adding a README!)
- Automatic styling
- Automatic linting

I really appreciate is how tools like {air}/{styler} and {jarl}/{flir} make this easy and repeatable.

Now let's talk about something that is a bit more of a skill that can't be automated: how to write readable code.

We will work on one script and discuss it, and some principles on good coding.

Overview

Duration 60 minutes
Questions

- Why go back over code you have already written?
- What makes a variable name good enough?
- What happens when two packages have a function with the same name?
- What are the steps of a review, and which of them can a machine do?
- How do I break a wall of code into readable pieces?
- Can I say what this code is for, and can I say it more simply?
- How do I review someone else's code without it going badly?

3.1 Clean as you go

There are two core ideas I'd like to get across this chapter:

> "Programs must be written for people to read, and only incidentally for machines to execute."
>
> – Harold Abelson and Gerald Jay Sussman, Structure and Interpretation of Computer Programs

Your code is only incidentally for computers - so we want it to be well written for us to understand, and protect our mental cycles!

The other idea is:

> You won't write your best code on the first pass.

Just like writing, I don't write my best code on the first pass. Your code gets better as you review it. The first time around you write code, is for getting it working, mostly. After that, you are improving readability, and expression.

To add another analogy into the mix: Professional kitchens clean as they go, because a tidy bench keeps your mind clear and because the alternative is chaos by service.

So a best practice to add to your learning is: when you finish a piece of code, go back over it briefly, and focus on the following:

1. Fix the style (use air or styler) - automated with saving :)
2. Read through it and add any notes to come back to later
3. Check the names of things - can they be improved?
4. Group the similar parts together
5. Review your comments - comment your code as little (and as well) as possible
6. Remove the "wilted lettuce"[1].

   - The commented-out chunks of code you **might just come back to**
   - You probably won't come back to these bits
   - If one really matters, write a comment saying why, rather than leaving it lying there.

7. Run a linter on the code

Now, you can't do a full review of your code, all the time.

But **doing some of it** will make your code better - and you'll get faster at it the more you practice.

Let's talk through these review practices by reviewing an example script.

[1]:Credit goes to Miles McBain for this name

3.2 The script

Imagine that someone hands you a script. It runs, it makes a plot at the end, and they would like another year of data added to it. Today, ideally.

Here's the script! Don't read it closely yet. Scroll through it. What do you notice?

```
library(readxl)
library(janitor)

# Read in the raw data
data <- read_excel(
  path = "data/Education and work, 2023, Datacube 2 (Table
   ↪  11).xlsx",
  sheet = "2014",
  skip = 4,
  n_max = 32,
  # use janitor::make_clean_names to turn `15-19 years` to
   ↪  "x15_19_years"
  .name_repair = make_clean_names
)

# sum(is.na(data))
# nrow(data)

# clean up the silly names from excel
names(data)
the_names <- names(data)
the_names[1] <- "state_territory"
the_names2 <- the_names

library(dplyr)
library(purrr)
# subset the data down to the number of educated people
 ↪  section
data
data_subset <- data %>%
  slice(4:11) %>%
  set_names(the_names2)


# sum(is.na(data_subset))
# nrow(data_subset)

data_subset

library(tidyr)
data_studying <- data_subset %>%
  pivot_longer(
    cols = -state_territory,
```

```
    names_to = "age_group",
    names_prefix = "x",
    names_pattern = "(.*)_years",
    values_to = "n_studying",
    values_transform = as.numeric
  ) %>%
  arrange(
    state_territory,
    age_group
  ) %>%
  # remove larger 15_24/64/74 age bands
  filter(
    age_group ≠ "15_24",
    age_group ≠ "15_64",
    age_group ≠ "15_74",
    age_group ≠ "18_24",
    age_group ≠ "25_64"
  )

# names(data_studying)

data_studying
# Population in age groups
data_population <- data %>%
  slice(24:31) %>%
  set_names(the_names2)

data_population <- data_population %>%
  pivot_longer(
    cols = -state_territory,
    names_to = "age_group",
    names_prefix = "x",
    names_pattern = "(.*)_years",
    values_to = "population",
    values_transform = as.numeric
  ) %>%
  arrange(
    state_territory,
    age_group
  ) %>%
  # remove larger 15_24/64/74 age bands
  filter(
    age_group ≠ "15_24",
    age_group ≠ "15_64",
    age_group ≠ "15_74",
    age_group ≠ "18_24",
    age_group ≠ "25_64"
  )

library(forcats)
# combine the data
# I guess it's only a study rather than the true numbers?
```

```
data_joined <- data_studying %>%
  left_join(data_population, by = c("state_territory",
  ↪  "age_group")) %>%
  mutate(
    age_group = as_factor(age_group),
    prop_studying = n_studying / population,
    year = 2014
  ) %>%
  relocate(
    year
  )

data_joined

library(ggplot2)
ggplot(data_joined, aes(x = prop_studying, y =
↪  state_territory)) +
  geom_col() +
  facet_wrap(~age_group)
```

This is a real script of mine, from a project called ozed that I used for an example project when teaching.

The goal of the analysis was to look at how many Australians are studying, by age group and by state, using data from the Australian Bureau of Statistics.

I want to be fair to it, because it is not **bad** code.

It runs. It's formatted. But I think we can do better.

Let's review the code, following our list from above:

1. Fix the style (use air or styler) - automated with saving :)
2. Read through it and add any notes to come back to later
3. Check the names of things - can they be improved?
4. Group the similar parts together
5. Review your comments - comment your code as little (and as well) as possible
6. Remove the "wilted lettuce"[2].

- The commented-out chunks of code you **might just come back to**
- You probably won't come back to these bits
- If one really matters, write a comment saying why, rather than leaving it lying there.

7. Run a linter on the code

[2]:Credit goes to Miles McBain for this name

3.3 Bad, better, and good variable names

> There are only two hard things in computer science: cache invalidation and naming things.
>
> – Phil Karlton[3]

It's comforting to know naming things is genuinely hard. I think we can sometimes let perfect become the enemy of good so when it comes to names - remember: **It is OK to have OK names**

> An OK name is better than a bad name An OK name is easier than a great name Use OK names

How do give things OK names?

> Say what it holds, in two or three words, joined by _. If a better name shows up later, rename it then.

If we can aim to make the names a little better - even just "OK", then that's good progress.

Here's a little demo of some names of four things:

3.4 data frame

```
# bad
d <- read_csv("data/penguins")
# OK
penguins <- read_csv("data/penguins")
# good
penguins_raw <- read_csv("data/penguins")
```

3.5 data filtered

```
# bad
df2 <- d ▷ filter(island == "Biscoe")
# OK
penguins_filtered ▷ filter(island == "Biscoe")
# good
penguins_biscoe ▷ filter(island == "Biscoe")
```

[3]:Phil Karlton was a principal developer at Xerox PARC, DEC, Silicon Graphics and Netscape. Martin Fowler has written up the provenance of the quote, which is more interesting than you'd expect. Nobody has ever found where Karlton first said it, and it seems to have travelled by word of mouth from about 1996.

3.6 model

```
# bad
m <- lm(body_mass ~ species, data = df2)
# OK
model <- lm(body_mass ~ species, data = penguins_filtered)
# good
lm_mass_by_species <- lm(body_mass ~ species, data =
 ↪  penguins_biscoe)
```

3.7 plot

```
# bad
p1 <- ggplot(df2, aes(x = body_mass, colour = species)) +
 ↪  geom_boxplot()
# OK
mass_plot <- ggplot(penguins_filtered,
                    aes(x = body_mass, colour = species)) +
    geom_boxplot()
# good
boxplot_mass_species <- ggplot(penguins_biscoe,
                               aes(x = body_mass, colour =
                                ↪  species)) +
    geom_boxplot()
```

- The bad data propagates throughout the process - it becomes increasingly harder to know what you are dealing with when modelling and plotting
- `penguins_filtered` doesn't tell me *which* filter. I am OK with that because it tells me: "this is the penguin data, and it has been filtered".
- Notice the gap between bad and OK is much wider than the gap between OK and good.
- It's OK to aim for OK.

As you practice giving OK names to things, you'll get more practice at naming, and get better. But don't put too much into it.

3.7.1 Changing names

A name isn't a commitment! You can rename easily, and there's some good tricks.

I highly recommend doing

- "Rename in scope" (Cmd / Ctrl + Shift + Alt + M) with your cursor on the name.
  - Note that this is different to find + replace!

You can search for this in the command palette if you forget.

> **Your Turn: rename the script**
>
> The script is `10-hard-to-read/analysis.R` in the course repo, or copy it out of the listing above.
> Most of its names are already OK. `data_studying`, `data_population` and `data_joined` all tell you what they hold, and I would leave them alone. Three are worth five seconds each:
>
> - `data`
> - `data_subset`
> - `the_names2`
>
> 1. Rename those three. Aim for OK, not great.
> 2. Use **Rename in Scope** (`Cmd / Ctrl + Shift + Alt + M`) or `F2`, rather than find and replace. Find and replace on a name like `data` will reach into `data_studying` and `data_subset` too.
> 3. Read the script again afterwards. How does it feel?
> 4. `the_names2` is the interesting one. What *is* it, and why are there two?

> **What `the_names2` turned out to be**
>
> Here are those three lines on their own:
>
> ```
> the_names <- names(data)
> the_names[1] <- "state_territory"
> the_names2 <- the_names
> ```
>
> The third line copies the second variable into a third one and never touches it again. `the_names2` **is** `the_names`, and the only reason it exists is that somebody was not sure whether they would need the original.
> The `2` is what hid it. A name ending in a number tells you there is another one somewhere, and nothing else, so you stop asking. Give it a real name, or delete the line.
> While you are here: `data` is worth renaming for a second reason. There is already a `data()` function in `{utils}`, so naming your data frame `data` shadows it - which is the same hazard as When two packages collide 3.8, made by hand.

3.8 When two packages collide

So far this has been about names you chose. This one is about a name you didn't.

Look back at the script. It has six `library()` calls scattered through it,

and further down it uses `filter()`.

Which `filter()`?

Do you know what this message here is telling us?

```
library(dplyr)
```

```
#>
#> Attaching package: 'dplyr'
#>
#> The following objects are masked from 'package:stats':
#>
#>     filter, lag
#>
#> The following objects are masked from 'package:base':
#>
#>     intersect, setdiff, setequal, union
```

R is telling you that there are now two functions called `filter()`, two called `lag()`, and that when you type one of those names it will hand you the `{dplyr}` one, because `{dplyr}` was attached most recently.

Which means the meaning of your code depends on the order of the `library()` calls at the top of your script. And that's a strange thing to be true.

Let's quickly show that as it hits you - does this look familiar?

`{MASS}` also has a `select()`:

```
library(dplyr)
```

```
Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(MASS)
```

```
Attaching package: 'MASS'
```

```
The following object is masked from 'package:dplyr':

    select
```

```
starwars ▷ select(name, height)
```

```
Error in `select()`:
! unused arguments (name, height)
```

An error, straight away. Annoying. Familiar?

> **!** This is why library() order is not a style question
>
> Two scripts with the same `library()` but in a different order can give different answers. You also need to consider if you've got another `library()` call in the same session - a reason to make sure you have blank slate settings in Project organisation 1.

3.8.1 The `conflicted` pkg to the rescue!

The {conflicted} package is built for this!

Instead of silently picking the last package attached, R refuses to pick at all:

```
library(conflicted)
library(dplyr)
library(MASS)
starwars ▷ select(name)
```

```
Error:
! [conflicted] select found in 2 packages.
Either pick the one you want with `::`:
* MASS::select
* dplyr::select
Or declare a preference with `conflicts_prefer()`:
* `conflicts_prefer(MASS::select)`
* `conflicts_prefer(dplyr::select)`
```

An error, at the line that is ambiguous, naming both candidates and telling you how to fix it. I think that is about as good as an error message gets.

You then say which one you meant, once, at the top of the script:

```
library(conflicted)
library(dplyr)
library(MASS)

conflicts_prefer(dplyr::select)
```

```
[conflicted] Will prefer dplyr::select over any other package.
```

```
starwars ▷ select(name)
```

```
# A tibble: 87 x 1
   name
   <chr>
 1 Luke Skywalker
 2 C-3PO
 3 R2-D2
 4 Darth Vader
 5 Leia Organa
 6 Owen Lars
 7 Beru Whitesun Lars
 8 R5-D4
 9 Biggs Darklighter
10 Obi-Wan Kenobi
# i 77 more rows
```

This is a nice example of code that is documenting the reasoning.

You can also get {conflicted} to do a little reporting for you:

```
conflict_scout()
```

```
3 conflicts

* `filter()`: dplyr and stats

* `lag()`: dplyr and stats

* `select()`: dplyr
```

Going deeper: why not just write dplyr::select() everywhere?

You can, and some people do. It removes the ambiguity completely, and it is common practice in R package development.
I find in an analysis, it is a bit of noise. I have to look at this code a lot! A `{ggplot2}` plot block written that way is `ggplot2::` down the left hand side of every line, and the shape of the code disappears behind a prefix that is identical on every row.
`{conflicted}` gets you the same safety without the tax, because the machine does the checking instead of you.
So should you use `pkg::fun()`? Maybe - here's some thoughts on where it might be more worth it.

- A package you use once for one utility, and don't otherwise need attached.

- Deliberately disambiguating at a single call site.
- In R package development

Your Turn

1. Run `library(conflicted)` and then attach the packages your current project uses. What does `conflict_scout()` say?
2. Pick one conflict. Which function were you actually getting?
3. Add `library(conflicted)` as the first line of one of your scripts and run it. Did anything break? Anything that breaks was already ambiguous.

3.9 A guide to reviewing code

The machine reviews the mechanics, the people review the thinking.

We talked about using the automated checks in air/styler and jalr/flir earlier - this means when you review code now, your attention is not on:

- spacing
- a misspelled comment, or
- `any(is.na(x))`

Apply these tools!

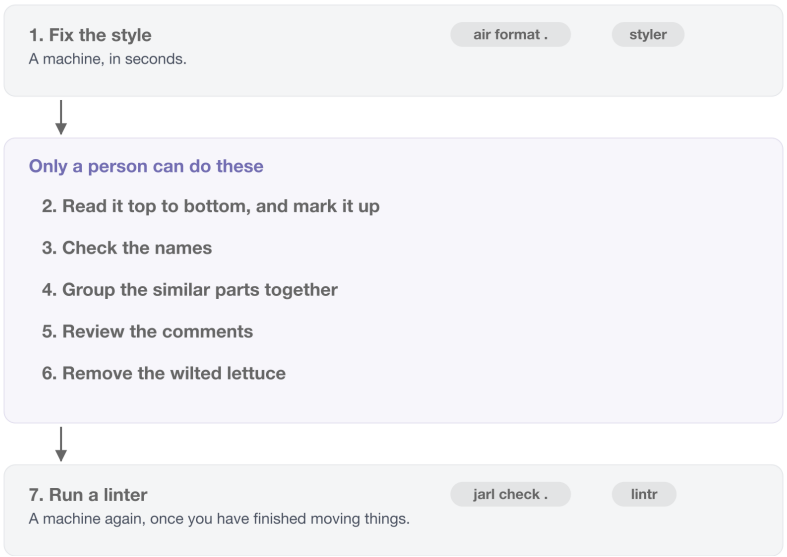


Figure 3.1: The machine does the first thing and the last thing. Everything in between is yours.

3.9.1 1. Fix the style

`air format .`, or `styler`. If you set up format-on-save back in Code style and readability 2, this has already happened and you can move on.

The other thing worth doing before you read a line: **run it from a clean session.** Restart R, run from the top. If it doesn't run, nothing else matters yet.

The linter comes at the *end*, in step 7, and not here. That is deliberate - you are about to move code around, and there is no sense linting a layout you are about to change.

> Going deeper: if what you are reviewing is a package
>
> Everything above works on a folder of scripts. If you are reviewing an R package, there is a tool that checks the habits a package is supposed to have:
>
> ```
> goodpractice::gp()
> ```
>
> It runs `R CMD check`, a set of linters, and a pile of structural checks - whether there is a README, whether the DESCRIPTION is well formed, whether functions are too long or too complex, whether there are tests.
> It's down here rather than in the list above because most of what it checks only exists in a package. Pointed at an analysis project, the great majority of its checks have nothing to look at.
> Excellent tool, wrong shape for the code most of us review day to day. Worth knowing about for the day you write a package.

3.9.2 2. Read it top to bottom, and mark it up

Read the whole thing without fixing anything.

The urge to fix the first thing you see is strong. Resist!

On this pass you're only getting the shape of the thing, and collecting the places where the code caught you: where it didn't quite make sense, where something was unclear, where you had to read a line twice.

Mark them with a word you can search for. I use two: TODO and NOTE

```
# TODO: this filter drops 400 rows and I do not know why
# NOTE: same block as line 40, with different columns
```

`TODO` for something that needs doing, `NOTE` for something you want to remember but which might turn out to be fine.

This beats holding it in your head for two reasons.

You can find them all again with a search. Writing the note is cheaper than fixing, so you keep reading instead of disappearing down the first rabbit hole.

> What you are marking: code smells
>
> A **code smell** is a bit of code that is not wrong, but that makes you uneasy.
> The word is deliberate. It's the feeling of opening the fridge and knowing something in there has gone off before you have worked out what.
> Smells are useful because they're cheap. You don't have to prove anything to act on one - you notice that a chunk is doing three things, or that a block has been copied twice with one number changed, and you write a `NOTE`. That's all this step is.
> Refactoring, in step 7, is what you do about them. Marking is not fixing.
> For a great introduction, watch Jenny Bryan's "Code smells and feels".

Steps 3 to 6 are the second pass, over the marks you just made.

3.9.3 3. Check the names

We did this one at length in Bad, better, and good variable names, because it is the step with the best return for the effort.

It sits here in the list so you know where it falls in the pass: after you have read the thing and marked it up, before you start moving code around. Renaming while you are still working out what the code does is how you end up with a confidently wrong name.

3.9.4 4. Group the similar parts together

The question to keep asking:

> How do I break this into pieces I can reason about one at a time?

Here is our script with every line coloured by the chunk it belongs to.

Figure 3.2: Five ideas. The package colour turns up in five separate places, which is the whole argument for grouping like with like.

Five ideas in thirty-odd lines. And the orange band, the `library()` calls, turns up five separate times on the way down.

Going deeper: why chunking works

Here is a phone number:

`1-8-0-0-1-3-1-0-8-6`

Ten digits. Try to hold them in your head. Now try this:

`1800 131 086`

Same ten digits, and suddenly it's easy. Nothing about the information changed. Only the **chunking** did.
This is not a party trick. Working memory holds roughly **seven, plus or minus two** chunks at once, from George Miller's 1956 paper "The magical number seven, plus or minus two". Memory isn't limited by the amount of information, but by the *number of chunks*.
So make bigger chunks and you hold more. Thirty lines you can't hold. Five named ideas you can.

3.9.4.1 Doing it to the script

Back to our ninety lines, now with the names fixed.

Read it once, and every time the *subject* changes, put in a blank line and a comment saying what just finished.

Don't fix anything. You're drawing lines, not editing.

Here is what I get:

```
# 1. read the raw spreadsheet for one year
# 2. fix the column names Excel gave us
# 3. pull out the education rows, make them long, drop the
 ↪  wide age bands
# 4. pull out the population rows, make them long, drop the
 ↪  wide age bands
# 5. join them, and work out the proportion studying
# 6. plot it
```

Six ideas!

Run the code - look at the input, look at the output Sometimes you can undersatnd the inputs and outputs without understanding the nitty gritty.

Now look at what falls out, for free.

Chunks 3 and 4 are the same thing. Same `slice()`, same `pivot_longer()`, same `arrange()`, same five-line `filter()`. The only differences are which rows get sliced and what the value column is called. Forty lines doing one idea twice.

The change they asked for lives in chunk 1. Another year is another sheet in the same workbook, and nothing after that first chunk cares which year it is looking at. But the year is baked into `sheet = "2014"`, and `educated_2014`, and `population_2014`, and `year = 2014` - so adding one currently means copying eighty lines and editing five numbers inside them.

The six chunks help identify these factors

3.9.4.2 Then put like with like

Once you can see the chunks, tidy them:

- All `library()` calls at the top.
- All function definitions together.
- If a variable is defined at line 10 and first used at line 150, move them closer.
- Chunks that do similar things should sit near each other.
- Add the commented headers, so the sections are visible without counting lines.
- If a section is getting long, should it be its own script?

None of this changes what the code does. All of it changes how much you've got to hold in your head.

Two questions worth asking here: **is the running order obvious** to someone who has never seen it, and **can you reason about each**

chunk on its own, or do you need the whole thing in your head at once?

> Where this script goes next
>
> Those six comments are a to-do list. Each line is a function waiting to be given a name, and the two identical steps are one function called twice.
> We pick the script up again in **?@sec-writing-functions** and take it the rest of the way, until adding a year is one argument rather than eighty copied lines. The whole progression, one script per step, is in the ozed repo.

> Your Turn: chunk something else
>
> You have watched me do it. Now do it to something I have not touched. Use `07-needs-review/analysis.R` from the course repo, or better, one of your own from last year.
> Work the list in order:
>
> 1. **Fix the style.** `air format .` on it first.
> 2. **Read it and mark it up.** `TODO` and `NOTE`, fix nothing.
> 3. **Check the names.** Rename three. Aim for OK.
> 4. **Group the similar parts.** Where does one chunk end and the next begin? How many are there?
>
> Then two questions the chunking will answer for you:
>
> - Are any two of those chunks doing the same thing to different data?
> - If someone asked you to add August, how many lines would you have to touch?
>
> Those last two are where the work is, and you can answer both from your chunk comments alone.

3.9.5 5. Review the comments

Now read what the comments say, rather than reading past them.

The ones to delete are the ones that restate the code:

```
# read in the data
data <- read_excel(path)
```

The ones to keep are the ones that say *why*, which the code cannot:

```
# skip = 4 because the ABS puts four rows of notes above the
 ↪  header
data <- read_excel(path, skip = 4)
```

Our script has one of each. `# Read in the raw data` tells you nothing that `read_excel()` doesn't. `# use janitor::make_clean_names to turn 15-19 years to x15_19_years` is genuinely useful, and I would keep it.

Worth reading on this: comment your code as little (and as well) as possible.

3.9.6 6. Remove the wilted lettuce

The commented-out code you might just come back to.

Our script has four of them:

```
# sum(is.na(data))
# nrow(data)
# sum(is.na(data_subset))
# names(data_studying)
```

Those are someone checking their work as they went, which is a good thing to have done and not a thing to keep. They are noise now, and worse, a reader has to decide whether each one matters before ignoring it.

Delete them. If one really does matter, write a comment saying why it is there, rather than leaving a dead line lying around.

If you are worried about losing something: that is what version control is for, and it is much better at remembering than a commented-out line is.

3.9.7 7. Run a linter

`jarl check .`, or `lintr::lint_dir()`.

Last, not first. You have just moved code around, deleted lines and renamed things, so this is the sweep that catches what you disturbed on the way through.

> Going deeper: if what you are reviewing is a package
>
> Everything above works on a folder of scripts. If you are reviewing an R package, there is a tool that checks the habits a package is supposed to have:
>
> ```
> goodpractice::gp()
> ```
>
> It runs `R CMD check`, a set of linters, and a pile of structural checks - whether there is a README, whether the DESCRIPTION is well formed, whether functions are too long or too complex, whether there are tests.
> Most of what it checks only exists in a package. Pointed at an

> analysis project, the great majority of its checks have nothing to look at.
> Excellent tool, wrong shape for the code most of us review day to day. Worth knowing about for the day you write a package.

3.10 Can you say what it is for?

The seven steps tidy code up. This is the deeper question underneath them, and it is the one that actually makes code readable.

3.10.1 Expression: can you follow it?

Now go back to the `TODO`s and `NOTE`s from step 2.

For each one: **can you say what the code is for, in a sentence?**

Here are two versions of the same calculation. Both give `0.27`, `0.33`, `0.81`.

3.11 Hard to say

```
d3 <- d2[d2$Month %in% c(5, 6, 7) & !is.na(d2$Ozone), ]
d3$f <- d3$Ozone > 30
res <- tapply(d3$f, d3$Month, mean)
```

3.12 Easy to say

```
prop_high_ozone <- airquality ▷
  filter(!is.na(Ozone), Month %in% 5:7) ▷
  group_by(Month) ▷
  summarise(prop_high = mean(Ozone > 30))
```

The second one has a sentence: *the proportion of days with high ozone, by month.*

For the first one I have to work out what `f` is, then what `tapply` is splitting by, then what `res` holds, and only then can I say the sentence. Same answer, and I had to run it in my head to get there.

If you can't say the sentence, here are two things to look at.

3.12.0.1 What does it need, and does it get used?

Two questions for any chunk:

1. **what does this need in order to run**, and
2. **does it get used?**

```
penguins_raw <- read_csv("penguins.csv")                    ①
penguins <- clean_names(penguins_raw)

n_total <- nrow(penguins)                                   ②
n_missing <- sum(is.na(penguins$body_mass_g))
pct_missing <- n_missing / n_total

mass_summary <- penguins |>                                 ③
  group_by(species) |>
  summarise(mass_mean = mean(body_mass_g, na.rm = TRUE))

temp <- mass_summary |> arrange(desc(mass_mean))            ②

ggplot(mass_summary, aes(species, mass_mean)) +             ③
  geom_col()
```

① Used further down. Fine.
② **Never mentioned again.** Four objects, computed and abandoned.
③ Used. This is the actual work.

Four of the eight objects in that block are never used again.

Usually it means somebody explored, found their answer, and left the exploring in.

Sometimes it means a result was supposed to be used and quietly isn't.

A chunk with many inputs and many outputs is usually several jobs that happen to be sitting next to each other.

3.12.0.2 Is there a simpler form?

Sometimes code does something perfectly reasonable in a roundabout way, and R already has the direct version.

This can happen with statistics.

3.13 Mean

```
sum(x) / length(x)

mean(x)
```

3.14 Variance

```
sum((x - mean(x))^2) / (length(x) - 1)

var(x)
```

3.15 Standard deviation

```
sqrt(sum((x - mean(x))^2) / (length(x) - 1))

sd(x)
```

3.16 Counting rows

```
nrow(d[d$mass > 4000, ])

sum(d$mass > 4000)
```

3.17 Lengths of a list

```
sapply(my_list, function(i) length(i))

lengths(my_list)
```

Each pair gives the same answer. The first one says how, the second one says what. We want the reader to know thw what.

You can't spot these unless you happen to know the shortcut, which is a bit hard - it takes experience! So don't go hunting. Just notice when a line feels like more machinery than the idea deserves, and go and look.

3.17.1 Re-expression: can you say it more simply?

The last question is the useful one.

Can you re-express the intention of the code, and keep the output the same?

That's **refactoring**: changing code while keeping its behaviour.

It's like giving a car a service, or replacing parts that have worn. The car drives the same, it just does it more reliably.

This is where everything you marked gets cashed in. The copied block becomes a function. The four abandoned objects get deleted.

Then you run it again and check the output hasn't moved.

That check isn't optional. It's very easy to tidy something into a different answer.

> **Your Turn**
>
> 1. Run steps 1 to 3 on `07-needs-review/analysis.R`. Actually run `air` and the linter, don't just read about them.
> 2. It still is not easy to follow. Mark three places where it caught you, with `TODO` or `NOTE`.
> 3. Pick one and do step 5. Re-express it, then check the output has not changed.

3.18 Reviewing someone else's code

When you review someone else code, we apply the same ideas as above, some some social pleasantries.:

1. **Ask what they want first.**

- *what kind of feedback are you looking for?* Someone who wants to know whether the statistics are right does not want thirty comments about variable names.
- *what is this code meant to do?* Find the intention before you review the implementation, because half of what looks like a mistake turns out to be a decision you did not have the context for.

2. **Review the code, not the person.**

- "This function is doing three things" is useful. "You always overcomplicate things" is not.

3. **Say what is good, specifically.**

- This is not politeness. If you only ever name problems, people learn what to avoid and never what to aim for.

4. **Run it, don't just read it.**

- Reading tells you whether code is comprehensible; running it on something unfamiliar surfaces the assumptions the author did not know they were making.

> **When it breaks and you cannot see why**
>
> Cut it down. Remove everything not needed to make it fail, one piece at a time, until what's left is small enough to see through. That is a review technique, and it is also exactly how you make a **reprex**. We come back to it in Writing a reprex / getting unstuck.

If you want to see this done at full scale

Open software peer review is the most developed version of this practice, and it is all public. rOpenSci's reviewer guide sets out what a reviewer is asked to look at, and their review template is the checklist itself. Reviews happen in the open on GitHub and are signed, so you can read hundreds of real ones.
Two things there are worth stealing for a small analysis project. Reviews are explicitly **non-adversarial**, because the goal is better software rather than a verdict. And reviewers are asked about things a checklist cannot capture, such as whether the argument names read well and autocomplete sensibly.
That last one should sound familiar from the naming section.

If we can see your code, we can trust it

Version control is not part of this course. It has its own.
But the single biggest improvement to reviewability is putting your code somewhere it can be seen, with a history. Review is much easier when the reviewer can see not just what the code is, but how it got that way.

Your Turn

1. Pair up. Swap the version of `analysis.R` you tidied earlier.
2. Before reading it, ask the two questions: what feedback do they want, and what is the code meant to do?
3. Give three pieces of feedback. At least one should be something that is working well.

If you want a longer one

`08-curly-code/penguin-report.R` in the exercises repo is the same idea at greater length. It runs, it produces a report, and `lintr::lint()` finds 125 things in it across 13 rules before you get to anything a person has to see.
Work the five steps on it end to end. The automated pass takes about a second, and everything interesting happens after that.

Read more

Parts of this chapter are adapted from my blog post, How to get good with R.

Links

- https://r-best-practices.njtierney.com
- R: A Language for Data Analysis and Graphics
- Quarto: callout blocks

- https://github.com/njtierney/rbestpractices
- https://github.com/njtierney/rbp-exercises
- https://cran.r-project.org/
- RStudio
- Positron
- palmerpenguins
- R version 4.5.0 is out
- `basepenguins`
- Air
- Jarl
- Etienne Bacher
- “Workflow”
- RStudio, what and why
- “quarto for scientists”
- course exercises
- Posit’s RStudio guide to the Command Palette
- “Visual Studio Code - Open Source”
- Posit’s article on the topic
- “PNG”
- “Project-oriented workflow”
- “The R Proj File”
- {here}
- Common sense approaches to sharing tabular data alongside publication
- “Introduction to git and github”
- git
- The files chapter of the Tidyverse style guide
- “How to name files like a normie”
- “Good enough practices in scientific computing”
- Hadley Wickham
- Lionel Hutz
- The Simpsons
- Workflow: code style
- kerning
- style chapter of Advanced R, 1st edition
- Tidyverse style guide
- files chapter
- Code formatters
- `{styler}`
- Wikipedia page for lint
- `{lintr}`
- `{flir}`
- How to get good with R
- Structure and Interpretation of Computer Programs
- comment your code as little (and as well) as possible
- Miles McBain
- ozed
- Australian Bureau of Statistics
- cache invalidation
- Phil Karlton
- Martin Fowler has written up the provenance of the quote

- the listing above
- the script
- {conflicted}
- Jenny Bryan's "Code smells and feels"
- Bad, better, and good variable names
- "The magical number seven, plus or minus two"
- ozed repo
- Writing a reprex / getting unstuck
- rOpenSci's reviewer guide
- review template
- How to get good with R